PLANO CANÔNICO MACRO

PDF 1 de 7 — Base oficial para substituição do motor mecânico + motor cognitivo atual por uma arquitetura LLM-first com governança, memória estruturada e policy engine mínimo.

Objetivo desta peça

Definir a visão macro, a arquitetura alvo, a estratégia de migração, os riscos principais, o cronograma executivo e a ordem oficial dos documentos da implantação.

Status	Canônico v1.0
Escopo	Substituição controlada da arquitetura conversacional e decisória da Enova
Modelo alvo	LLM-first + estado estruturado + policy engine + telemetria + multimodal
Leitura recomendada	Este PDF deve ser lido antes do contrato de implantação e do plano tático

Tese central: a Enova deve sair de uma arquitetura centrada em trilhos rígidos e múltiplos motores sobrepostos para uma arquitetura em que o LLM seja o motor principal da conversa, enquanto a governança fica presa em políticas explícitas, memória estruturada, validações mínimas e telemetria forte.

1. Decisão estratégica

A decisão não é “trocar um motor por outro” de forma cega. A decisão correta é migrar de uma arquitetura de múltiplos motores e lógica espalhada para um sistema com centro semântico único, governado por regras explícitas, estado estruturado e auditoria forte.

- O LLM passa a ser o motor principal da conversa, interpretação e condução contextual.
- A governança deixa de ficar escondida em trilhos e hacks acumulados, e passa a viver em um catálogo de regras explícitas.
- O estado do cliente deixa de ser apenas histórico textual e passa a ser uma memória operacional estruturada.
- A telemetria vira instrumento obrigatório de validação, regressão, rollout e prova de segurança.
- O multimodal deixa de ser exceção e passa a ser capacidade nativa da plataforma.

2. O que será substituído e o que será preservado

A migração não deve jogar fora o ativo mais valioso já construído: as regras de negócio e os aprendizados reais do funil. O que muda é a forma de execução, não a inteligência acumulada.

Bloco	Definição
Será substituído	Motor mecânico rígido como centro de decisão; motor cognitivo paralelo de casca; dependência excessiva de trilhos fixos para conversa; expansão cara para áudio, sticker e imagem.
Será preservado	Regras reais do programa e do funil; taxonomia de perfis; telemetria já provada; critérios de elegibilidade; linguagem comercial segura; política de não prometer aprovação.
Será convertido	Stages e gates atuais serão reexpressos como objetivos, facts, pendências, regras obrigatórias, bloqueios e critérios de conclusão.
Será ampliado	Capacidade multimodal, expansão futura de regras, memória persistente mais limpa, avaliação por bateria de casos e rollout controlado.

3. Arquitetura-alvo

A arquitetura canônica do novo modelo é composta por camadas pequenas, claras e auditáveis. O LLM conversa livremente; a decisão, o avanço e a conformidade são controlados por estruturas externas e explícitas.

Bloco	Definição
Camada 1 — Agent Core	Prompt constitucional do agente, identidade da Enova, objetivos por etapa, tom comercial permitido, limites do que não pode falar e contrato de saída estruturada.
Camada 2 — Conversation Runtime	Gerencia o turno, reúne contexto, injeta memória resumida, chama o modelo, recebe resposta natural + payload estruturado e prepara o próximo passo.
Camada 3 — State Store	Guarda facts confirmados, pendências, conflitos, histórico resumido, origem do fato, timestamps, flags de risco e objetivo atual do caso.
Camada 4 — Policy Engine	Aplica regras obrigatórias, bloqueios, confirmações, sugestões mandatórias e roteamentos, sem determinar a linguagem da conversa.
Camada 5 — Telemetria & QA	Registra sinais por turno, violações de regra, conflitos de memória, métricas por caso e evidências para shadow mode, rollback e hardening.
Camada 6 — Multimodal IO	Entrada e saída por texto, áudio, imagem, sticker e documentos, todos convergindo para a mesma governança.

4. Princípios canônicos de engenharia

Toda a implantação deve obedecer a princípios que evitem repetir o problema atual em formato novo.

- Conversa livre; governança presa.

- Nenhuma regra crítica pode depender apenas do texto do prompt.
- Todo avanço relevante deve ser explicável em facts + policies satisfeitas.
- Toda nova regra futura deve entrar por taxonomia e catálogo de policies, não por improviso.
- Toda mudança estrutural deve nascer primeiro em shadow mode ou teste controlado.
- Nenhum cutover acontece sem critérios de aceite e rollback definidos antes.

5. Modelo de estado estruturado

A substituição do “inferno do mecânico” exige que a memória deixe de ser implícita. O caso do cliente deve ser representado por um objeto operacional, versionado e auditável.

Bloco	Definição
Facts centrais	nome, nacionalidade, rnm_status, estado_civil, composição do processo, regimes de trabalho, rendas, dependente, restrição, CTPS, IR, canal preferido, intenção de visita.
Facts derivados	elegibilidade provável, necessidade de composição, bloqueio por RNM, necessidade de confirmação, risco documental, perfil de dossiê.
Pendências	informações obrigatórias ainda não confirmadas, perguntas mandatórias em aberto, validações faltantes.
Conflitos	dados contraditórios entre turnos ou fontes, pedindo confirmação controlada antes de avançar.
Objetivo atual	o próximo objetivo operacional do caso, não necessariamente um stage rígido, mas uma meta clara como “fechar composição” ou “confirmar IR do autônomo”.

6. Classes de policy

As regras devem ser escritas de forma declarativa, expansível e auditável. O mesmo motor deve suportar regras atuais e futuras sem virar outro monólito.

Bloco	Definição
Obrigatória	Algo que precisa acontecer antes de avançar. Ex.: autônomo exige pergunta sobre IR.
Bloqueio	Algo que impede continuidade. Ex.: estrangeiro sem RNM válido não pode seguir.
Roteamento	Define caminho operacional. Ex.: casado no civil implica processo em conjunto.
Sugestão mandatória	A conversa precisa oferecer a saída certa. Ex.: solo com renda baixa precisa ouvir composição antes de inviabilização.
Confirmação	Quando há ambiguidade, o caso não avança sem confirmação. Ex.: composição familiar x parceiro.
Compliance de fala	Restrições do que a Enova não pode afirmar, prometer ou omitir.

7. Macrocronograma oficial

A janela base de implantação recomendada é de 13 semanas. É uma estimativa séria, não otimista artificial. Dá para acelerar em partes, mas não é saudável prometer uma migração dessa profundidade em poucos dias.

Fases macro e janela-base de implantação

Fase	Nome	Semanas	Entrega principal	Gate de saída
0	Congelamento e diagnóstico	1	Mapa fechado do legado vivo, riscos e contrato de transição	Inventário aprovado
1	Contrato cognitivo canônico	1	Constituição do agente, taxonomia e regras	Contrato assinado

2	Estado estruturado v1	2	Modelo de memória, facts, pendências e conflitos	Persistência validada
3	Policy engine v1	2	Motor de regras obrigatórias e bloqueios	Casos críticos aprovados
4	Orquestrador LLM-first	2	Loop de turno completo com saída estruturada	Turnos estáveis
5	Migração de trilhos prioritários	3	Topo + qualificação + renda + elegibilidade	Paridade funcional
6	Multimodal e docs	2	Áudio, imagem, sticker e leitura orientada	Canal multimodal controlado
7	Shadow mode, hardening e cutover	2	Rodada paralela, métricas e go-live	Go/no-go aprovado

8. Microetapas por fase

As microetapas abaixo definem o nível mínimo de detalhamento executivo desta frente. O detalhamento operacional fino de cada fase será aprofundado nos próximos PDFs da série.

Fase 0 — Congelamento e diagnóstico (Semana 1)

- Inventariar o que ainda está vivo no motor mecânico e no cognitivo atual.
- Separar regra real de acúmulo técnico, workaround e lixo estrutural.
- Mapear os casos críticos do funil que não podem regredir.
- Fechar baseline de telemetria e KPIs de comparação.
- Definir pacote oficial de rollback e convivência temporária.

Fase 1 — Contrato cognitivo canônico (Semana 2)

- Escrever a constituição do agente: identidade, escopo, tom, limites e missão.

- Transformar regras de negócio em linguagem declarativa e não ambígua.
- Definir formato de saída estruturada do modelo.
- Criar catálogo inicial de objetivos operacionais.
- Fixar regras de compliance de fala e de handoff.

Fase 2 — Estado estruturado v1 (Semanas 3 e 4)

- Projetar o schema de facts, pendências, conflitos e flags derivadas.
- Definir origem, confiança, timestamp e regra de atualização de cada fato.
- Criar memória curta do turno e memória operacional persistente.
- Implementar mecanismo de confirmação quando houver conflito.
- Validar persistência, reload e integridade do caso entre sessões.

Fase 3 — Policy engine v1 (Semanas 5 e 6)

- Converter regras prioritárias em policies testáveis.
- Implementar classes: obrigatória, bloqueio, roteamento, sugestão mandatória e confirmação.
- Criar priorização e resolução de conflito entre policies.
- Montar bateria de testes de casos críticos.
- Provar que o avanço não depende só do texto do modelo.

Fase 4 — Orquestrador LLM-first (Semanas 7 e 8)

- Montar loop do turno com input, contexto, memória, chamada ao modelo e validação.
- Encaixar saída natural + payload estruturado em um fluxo único.
- Registrar telemetria por turno e por decisão.
- Implementar fallback controlado quando output vier incompleto.
- Fechar contrato de erro seguro e reask objetivo.

Fase 5 — Migração dos trilhos prioritários (Semanas 9 a 11)

- Migrar topo de funil, identificação, composição e renda.
- Migrar regras de autônomo, IR, dependente, nacionalidade e restrição.
- Provar paridade funcional contra os casos históricos da Enova.
- Refinar tom conversacional sem violar policy.
- Preparar ponte para docs e pós-aprovação.

Fase 6 — Multimodal e docs (Semanas 12 e 13)

- Acoplar entrada por áudio e saída por áudio sob a mesma governança.

- Acoplar interpretação de sticker, imagem e documento como sinais do caso.
- Padronizar como mídia altera facts e pendências.
- Integrar leitura documental ao novo runtime.
- Provar que o multimodal não quebra disciplina operacional.

Fase 7 — Shadow mode, hardening e cutover (Semanas 14 e 15)

- Rodar legado e novo runtime em paralelo em amostras controladas.
- Comparar paridade, violações de policy, repetição de pergunta e estabilidade.
- Corrigir edge cases antes de abrir tráfego real.
- Executar checklist formal de go-live.
- Cortar para o novo modelo com rollback pronto e janela de observação intensiva.

9. Critérios de aceite macro

A migração só pode ser considerada bem-sucedida se cumprir critérios de qualidade, negócio e governança ao mesmo tempo.

- Nenhuma regra crítica do negócio pode ficar presa exclusivamente ao prompt.
- Casos prioritários precisam passar com paridade ou melhoria comprovada.
- O novo runtime não pode aumentar repetição burra, drift ou promessas indevidas.
- Telemetria precisa mostrar o porquê do avanço, bloqueio, reask ou handoff.
- O canal multimodal precisa respeitar a mesma governança do texto.
- Rollback precisa estar testado antes do cutover e não só “documentado”.

10. Riscos e contenções

Trocar dois motores por uma arquitetura nova é uma operação séria. O erro aqui não é ousar; o erro é ousar sem disciplina.

Riscos principais e resposta executiva

Risco	Impacto	Sinal de falha	Mitigação mandatória
Drift conversacional	Lead mal conduzido, respostas fora da política	Mudança de conduta para casos equivalentes	Saída estruturada + policy checks + testes de regressão
Memória inconsistente	Perguntas repetidas ou decisão errada	facts contraditórios ou perda de contexto	Modelo de estado com conflitos, timestamps

			e origem do fato
Regra crítica não aplicada	Erro de negócio em renda, IR, estado civil	caso crítico passa sem bloqueio	Classe de regra obrigatória com gate de avanço
Cutover prematuro	Quebra operacional em produção	queda de paridade e aumento de handoff manual	shadow mode, checklist de go-live e rollback canônico
Expansão sem contrato	Bagunça futura com regras soltas	cada nova regra nasce de forma diferente	taxonomia + catálogo de policies + contrato rígido de mudança

11. Ordem oficial dos documentos desta série

Esta série foi desenhada para que a implantação não saia do eixo. A leitura e execução devem seguir a ordem abaixo.

Bloco	Definição
PDF 1	Plano Canônico Macro — visão, arquitetura, cronograma, riscos e ordem dos documentos.
PDF 2	Contrato de Implantação — regras rígidas de execução, papéis, gates, rollback e critérios de corte.
PDF 3	Plano Tático de Execução — sequência operacional por sprint/fase, com dependências e testes.
PDF 4	Backlog Mestre — épicos, histórias, critérios de aceite, prioridades e riscos por item.
PDF 5	Playbook Operacional — rotina de trabalho, checklist de revisão, shadow mode, rollout e incidentes.
PDF 6	Taxonomia Oficial — facts, policies, objetivos, conflitos, eventos, métricas e classes de mídia.

PDF 7	Contrato Rígido de Ordem de Execução — macro e microetapas travadas, com gates por fase.

12. Fechamento executivo

Conclusão estratégica: a Enova já amadureceu o suficiente para deixar de ser refém do mecânico pesado e do cognitivo paralelo. O passo certo agora não é soltar o LLM; é colocá-lo no centro com uma espinha dorsal de políticas, memória e telemetria. Esse novo modelo reduz atrito, abre o multimodal, preserva a inteligência real do funil e cria uma base muito mais expansível para os próximos anos.

Próximo PDF recomendado: Contrato de Implantação — peça que vai travar a ordem de execução, responsabilidades, gates de aprovação, rollback e critérios de convivência entre legado e novo runtime.

CONTRATO DE IMPLANTAÇÃO

PDF 2 de 7 — Documento vinculante da migração do modelo atual para a arquitetura LLM-first com governança, estado estruturado, policy engine e telemetria canônica.

Função deste contrato

Amarrar o que pode, o que não pode, quais gates precisam existir, como a implantação deve ocorrer, quais evidências são obrigatórias e em quais condições a migração pode avançar, pausar, regredir ou entrar em rollback.

Status	Canônico v1.0
Natureza	Contrato executivo e técnico de implantação
Escopo	Substituição do motor mecânico + motor cognitivo atual por uma arquitetura unificada LLM-first
Pré-requisito	Leitura prévia do PDF 1 — Plano Canônico Macro
Efeito	Nenhuma frente pode executar fora da ordem, gates e evidências aqui definidos

Princípio jurídico-operacional do projeto: o LLM terá liberdade conversacional, mas jamais liberdade decisória irrestrita. Decisão operacional, persistência de fatos críticos, avanço de objetivo e desbloqueio de cenários sensíveis devem obedecer ao contrato de políticas, estado e validações mínimas aqui instituído.

Objeto do contrato

Este contrato regula a implantação do novo modelo ENOVA LLM-FIRST, cuja finalidade é substituir de forma controlada os motores atuais por uma arquitetura única em que o LLM conduz a interação, enquanto estado estruturado, políticas explícitas, validações mínimas e telemetria canônica preservam previsibilidade, segurança e auditabilidade.

Cláusulas-mestras e efeito operacional

Cláusula	Efeito
Cláusula 1 — Preservação de negócio	Toda decisão sensível do novo modelo deve preservar as regras de negócio consolidadas da Enova, mesmo que a forma de perguntar mude.
Cláusula 2 — Liberdade com governança	O agente pode variar fala, tom, ordem natural de acolhimento e tratamento multimodal, mas não pode descumprir política obrigatória.
Cláusula 3 — Estado como fonte de verdade	Fato confirmado deixa de depender do contexto textual cru e passa a viver em estado estruturado, auditável e reutilizável.
Cláusula 4 — Política explícita	Toda regra crítica deve existir em política declarativa, não escondida apenas em prompt.
Cláusula 5 — Cutover por provas	Nenhuma substituição total ocorre sem shadow mode, métricas mínimas, bateria de casos reais e gate formal de aprovação.
Cláusula 6 — Rollback obrigatório	Qualquer regressão relevante de elegibilidade, classificação, coleta de fatos ou segurança operacional obriga rollback imediato da frente afetada.

Não negociáveis da implantação

- Casado no civil continua sendo processo em conjunto, sem flexibilização conversacional que burle a regra.
- Autônomo sempre exige verificação de IR; a regra pode ser respondida com flexibilidade de fala, mas não pode ser ignorada.

- Renda solo baixa exige política de sugestão de composição antes de qualquer rota de desistência ou inelegibilidade apressada.
- Estrangeiro sem RNM válido não pode avançar como se estivesse regular.
- O agente nunca promete aprovação, valor de entrada, parcela ou imóvel sem as validações apropriadas.
- A migração não pode destruir a telemetria já conquistada; o novo modelo deve nascer com evidência melhor, não pior.
- Toda resposta natural deve continuar capaz de ser auditada em fatos, políticas acionadas, riscos detectados e próximo objetivo.

Escopo, exclusões e fronteira de mudança

Bloco	Incluído neste contrato	Fora do escopo imediato
Motor conversacional	Substituição do desenho mecânico/cognitivo por um orquestrador LLM-first governado	Refatorações cosméticas sem ganho operacional
Memória/estado	Modelo de facts, pendências, conflitos, objetivos e histórico resumido	Armazenar tudo sem taxonomia ou sem política de retenção
Policy engine	Regras declarativas, obrigações, bloqueios, gates e fallback de governança	Regex espaguete como eixo principal da decisão
Multimodal	Texto, áudio, imagem, sticker e documentos sob a mesma governança	Abrir todos os canais ao mesmo tempo sem shadow mode
Operação legado	Mapeamento do legado vivo e retirada gradual controlada	Apagar trilhos antigos sem prova de cobertura do novo

Entregáveis vinculantes por frente

Cada frente abaixo só é considerada concluída quando entregar o pacote inteiro: artefato técnico, critérios de aceite, evidências, smoke tests, plano de rollback e telemetria mínima.

Frente A — Contrato cognitivo e constituição do agente

- System prompt canônico em camadas: identidade, limites, políticas de fala, riscos e formato de saída.

- Manual de regras do funil e do programa convertido em política declarativa legível.
- Taxonomia de fatos, sinais, pendências, conflitos e objetivos.
- Matriz do que o agente pode responder livremente e do que exige validação/política.

Frente B — Estado estruturado e memória

- Schema do estado do lead com versionamento.
- Regras de confirmação, atualização, conflito e descarte de fatos.
- Política de resumo de contexto para turnos longos.
- Compatibilidade transitória com o estado legado enquanto durar a migração.

Frente C — Policy engine e guardrails

- Regras classificadas em obrigação, bloqueio, sugestão mandatória, confirmação e roteamento.
- Validador pós-resposta e pré-persistência.
- Registro das políticas acionadas por turno.
- Mecanismo de veto quando o LLM tenta descumprir uma regra crítica.

Frente D — Orquestrador de turno

- Pipeline único de entrada, interpretação, extração, validação, resposta e persistência.
- Saída estruturada padrão para qualquer canal.
- Fallbacks de governança para ambiguidade, conflito e ausência de resposta útil.
- Integração com telemetria e IDs de execução por turno.

Frente E — Multimodal e docs

- Entrada por áudio com transcrição e reconciliação de fatos.
- Saída por áudio controlada por política.
- Tratamento de sticker/imagem como sinal complementar e nunca como decisão cega.
- Leitura documental guiada por taxonomia, sem destruir o fluxo de negócio.

Gates obrigatórios de implantação

Gate	Entrada	Prova exigida	Responsável	Saída permitida	Rollback se falhar
G0	Inventário legado	Mapa do que está vivo, morto, crítico e sensível	Arquitetura	Assinar migração	Não iniciar implantação

G1	Contrato do agente	Prompt canônico + políticas + taxonomia aprovados	Produto/ Arquitetura	Abrir desenvolvimento	Reescrever contrato
G2	Estado v1	Schema e persistência validados em casos críticos	Engenharia	Subir policy engine	Manter legado
G3	Policy v1	Casos de obrigação/bloqueio/roteamento passando	Engenharia/ Negócio	Ligar orquestrador	Desligar policy nova
G4	Orquestrador v1	Turnos completos auditáveis e estáveis	Engenharia	Iniciar shadow mode	Recuar para piloto interno
G5	Shadow mode	Paridade mínima com casos reais + métricas	Operação/ Produto	Abrir tráfego controlado	Voltar a legado
G6	Cutover parcial	Risco aceito + monitoramento em tempo real	Direção/ Produto	Migrar frentes escolhidas	Reversão da frente
G7	Cutover total	Frentes estáveis + backlog crítico zerado	Direção	Desligar motores antigos por etapa	Reativar camada anterior

Critérios de aceite executivos

Uma frente só fecha quando cumprir simultaneamente os critérios abaixo:

- Critério funcional: a frente cumpre o objetivo declarado em casos normais e críticos.
- Critério de política: não viola nenhuma regra obrigatória da Enova ou do programa.

- Critério de telemetria: gera evidência suficiente para auditoria do turno e da decisão.
- Critério de regressão: não piora os cenários já estáveis no legado que continuam em uso.
- Critério de rollback: possui chave de desligamento claro, sem dependência obscura.
- Critério operacional: a equipe consegue entender por que o agente respondeu, perguntou, bloqueou ou sugeriu algo.

Janela executiva com prazos contratuais

A janela-base recomendada é de 13 semanas. Os prazos abaixo são contratuais para gestão da implantação; podem ser comprimidos somente se as provas forem mantidas integralmente.

Semana	Frente macro	Microetapas obrigatórias	Marco	Dependência
1	Congelamento	Inventário, classificação de risco, mapa de compatibilidade	G0 aprovado	Nenhuma
2	Contrato	System prompt, políticas, taxonomia, contrato de saída	G1 aprovado	G0
3-4	Estado	Schema, persistência, conflitos, resumos, compatibilidade	G2 aprovado	G1
5-6	Policy engine	Regras declarativas, veto, priorização, testes críticos	G3 aprovado	G2
7-8	Orquestrador	Loop completo, saída estruturada, telemetria por turno	G4 aprovado	G3
9-11	Migração	Topo,	Paridade mínima	G4

	prioritária	qualificação, renda, elegibilidade, docs iniciais		
12	Shadow mode	Rota paralela, monitoramento, ajuste fino	G5 aprovado	Paridade mínima
13	Cutover inicial	Migração controlada e observada	G6 aprovado	G5

Responsabilidade mínima por papel (RACI simplificado)

Papel	Responsabilidade mínima
Direção do projeto	Aprovar contrato, gates críticos, cutover e rollback
Arquitetura	Desenhar camadas, fronteiras, compatibilidade e política estrutural
Engenharia	Implementar estado, policy, orquestrador, integrações e telemetria
Produto/Negócio	Validar aderência às regras do funil, do programa e da operação real
Operação	Prover casos reais, validar linguagem e impacto na condução comercial
QA/Observabilidade	Conduzir shadow mode, métricas, regressão e rastreabilidade

Cláusula de incidentes, congelamento e rollback

Se o novo modelo causar regressão em classificação crítica, promessa indevida, bloqueio indevido, perda de fatos ou condução operacional insegura, a frente entra imediatamente em modo de contenção.

Modo de contenção significa: congelar novas expansões, preservar evidências, isolar o bloco causador, acionar rollback da frente e abrir diagnóstico formal antes de qualquer nova mudança.

Nenhum incidente grave deve ser tratado com tentativa e erro no escuro. A ordem obrigatória permanece: observação, diagnóstico, confirmação, correção cirúrgica, revalidação.

- Rollback pode ser parcial por frente; não precisa derrubar toda a arquitetura se o desenho modular estiver correto.
- Toda chave de recurso novo deve existir por feature flag, com possibilidade de desativação imediata.
- Telemetria e logs da falha devem ser preservados como prova e base de aprendizado.

Ordem rígida de execução deste pacote documental

- PDF 1 — Plano Canônico Macro: visão, arquitetura-alvo e mapa da migração.
- PDF 2 — Contrato de Implantação: documento presente; trava escopo, gates e prazos.
- PDF 3 — Plano Tático de Execução: tradução do contrato em frentes operacionais e microentregas.
- PDF 4 — Backlog Mestre: inventário priorizado de tarefas, dependências, critérios e risco.
- PDF 5 — Playbook de Operação e Gestão: rotina semanal, ritos, indicadores e governança viva.
- PDF 6 — Taxonomia Canônica: fatos, entidades, sinais, políticas, objetivos, conflitos e eventos.
- PDF 7 — Contrato Rígido de Ordem de Execução: sequência mandatória de implantação com gates finais e proibições formais.

Regra vinculante: nenhum documento posterior pode contrariar este contrato; só pode detalhar e operacionalizar o que aqui foi fixado.

Encerramento executivo

O presente contrato existe para impedir duas falhas opostas: a primeira é continuar refém de um mecânico excessivamente rígido e caro de manter; a segunda é trocar esse mecânico por um LLM solto, bonito e perigoso. O modelo aprovado neste pacote segue uma terceira via: liberdade conversacional com governança dura. Essa é a fundação oficial da substituição estrutural da Enova.

PLANO TÁTICO DE EXECUÇÃO

PDF 3 de 7 — Sequência executiva detalhada para desmontar o motor mecânico + motor cognitivo atual e implantar o novo modelo LLM-first com governança, estado estruturado, policy engine e cutover controlado.

Função desta peça

Traduzir o plano macro e o contrato de implantação em sequência de trabalho executável: frentes, semanas, microetapas, dependências, critérios de pronto, pacote de testes, evidências mínimas e ordem exata de avanço.

Status	Canônico v1.0
Natureza	Plano executivo e operacional
Leituras prévias	PDF 1 — Plano Canônico Macro; PDF 2 — Contrato de Implantação
Horizonte-base	15 semanas com buffer controlado de 2 semanas
Uso obrigatório	Toda execução, PR, branch, teste e gate deve obedecer a este encadeamento

Regra executiva central: o projeto não troca tudo de uma vez. Primeiro congela e mapeia, depois define contrato e estado, depois implanta policy engine, depois sobe o orquestrador LLM-first em sombra, depois migra fatias do funil por prioridade, e só no fim executa cutover progressivo com rollback pronto.

1. Estratégia operacional de execução

A implantação deve seguir uma lógica de substituição progressiva por camadas. O objetivo não é reescrever a Enova inteira em um único movimento, e sim trocar o eixo da decisão com o menor risco possível para atendimento, elegibilidade, coleta de fatos, docs e operação comercial.

Eixo	Diretriz tática
Modo de trabalho	Fatia por fatia, com isolamento por frente e validação antes de abrir a frente seguinte.
Unidade de entrega	Cada frente entrega contrato + código/infra + testes + telemetria + rollback.
Estratégia de risco	Shadow mode antes de cutover; canary antes de 100%; rollback por feature flag e roteamento.
Critério de avanço	Só avança quando a camada nova cobre a camada antiga com evidência suficiente.
Estratégia de legado	O que estiver vivo no legado só sai depois de cobertura comprovada da nova camada.

Princípios de execução

- Separar sempre arquitetura, estado, políticas, orquestração, canais e migração de fluxo. Misturar tudo na mesma frente aumenta drift e mata governança.
- Toda regra crítica do negócio deve nascer em política declarativa e também existir em suíte de casos de validação.
- Toda fase deve produzir um mapa explícito do que ainda depende do legado e do que já pode ser desligado.
- Toda etapa deve deixar uma trilha mínima de auditoria: qual regra rodou, quais fatos foram atualizados, qual objetivo foi escolhido e por que o agente respondeu daquele jeito.

2. Cronograma tático mestre

A tabela abaixo é o cronograma-base. O plano assume 15 semanas de execução principal e até 2 semanas de buffer para correções de cobertura, performance, canais ou hardening final.

Bloco	Semanas	Objetivo	Saída concreta	Dependência	Gate
T0 — Congelar e mapear	1	Fechar mapa do legado vivo	Inventário de fluxos, estados,	—	Inventário

			parsers, dependências e riscos		aprovado
T1 — Constituição do agente	2	Fechar contrato cognitivo + taxonomia + formato de saída	System prompt canônico + manual de fatos e objetivos	T0	Contrato assinado
T2 — Estado estruturado	2	Subir schema, persistência e reconciliação	Lead state v1 persistido e auditável	T1	Estado validado
T3 — Policy engine	2	Implementar regras obrigatórias e veto leve	Motor declarativo com testes críticos	T2	Políticas críticas aprovadas
T4 — Orquestrador LLM-first	2	Fechar ciclo de turno unificado	Entrada → interpretação → validação → resposta → persistência	T2 + T3	Turno estável
T5 — Migração do funil core	3	Migrar topo + qualificação + renda + elegibilidade	Paridade funcional dos fluxos prioritários	T4	Paridade comprovada
T6 — Docs e multimodal	2	Acoplar docs, áudio, imagem e sticker	Governança multicanal em sombra	T5	Canais controlados
T7 — Shadow, canary e cutover	1–2	Troca progressiva em produção	Go-live com fallback e rollback	T6	Go/no-go executivo

3. Detalhamento por frente e microetapas

T0 — Congelamento, inventário e mapa do legado vivo

Janela-base: Semana 1. Objetivo: Descobrir exatamente o que está rodando hoje, o que ainda é dependência real do mecânico, do cognitivo e da infraestrutura, e o que pode ser desativado depois.

Entregáveis da frente

- Mapa dos fluxos vivos do início ao pós-envio_docs, com regras de negócio e pontos de risco.
- Inventário dos estados persistidos, campos realmente usados, compatibilidades temporárias e resíduos de legado.
- Mapa de canais e superfícies: texto, docs, áudio potencial, imagem, sticker, endpoints e telemetria.
- Lista de regras que precisam virar política explícita no novo modelo.

Microetapas obrigatórias

1. Congelar mudanças estruturais no legado durante a janela de mapeamento; só entra hotfix comprovado.
2. Listar todo ponto onde decisão hoje depende de parser, regex, fallback, heurística ou branch de stage.
3. Classificar cada regra em: negócio, compliance, docs, UX, operação comercial, roteamento e exceção.
4. Montar uma matriz de risco: o que quebra elegibilidade, o que quebra tom, o que quebra docs, o que quebra telemetria.
5. Fechar um inventário de desligamento futuro: o que sai primeiro, o que precisa conviver e o que deve permanecer por compatibilidade.

Pacote mínimo de testes

- Smoke read-only sobre fluxos reais e históricos para provar que o mapa bate com o comportamento operacional.
- Validação com recorte de leads reais: solo, conjunto, autônomo sem IR, estrangeiro, composição e docs.
- Checklist de completude do inventário aprovado pelo gate técnico.

Regra de rollback da frente

Se o inventário não fechar ou houver divergência séria entre mapa e comportamento real, a implantação não avança para T1.

T1 — Constituição do agente e contrato cognitivo canônico

Janela-base: Semanas 2–3. Objetivo: Transformar o conhecimento da Enova em um contrato operacional claro para o LLM, com identidade, limites, objetivos, taxonomia e saída estruturada.

Entregáveis da frente

- System prompt canônico em camadas, sem ambiguidades centrais.
- Manual declarativo das regras do funil e do programa, separado de estilo de fala.

- Taxonomia oficial de facts, objetivos, pendências, conflitos, riscos e ações.
- Formato de saída padrão do agente para qualquer canal.

Microetapas obrigatórias

1. Separar o que é tom, o que é regra de negócio, o que é veto, o que é sugestão mandatória e o que é simples repertório de resposta.
2. Definir como o agente deve agir em perguntas fora do trilho sem perder o objetivo operacional do turno.
3. Definir comportamento em ambiguidade, contradição, cliente prolixo, cliente evasivo, cliente com áudio ruim e cliente insistente em preço/aprovação.
4. Fechar o contrato de saída: reply_text, facts extraídos, fatos atualizados, objetivo atual, próxima ação pretendida, needs_confirmation, políticas acionadas e bloqueios.
5. Documentar explicitamente o que o agente nunca pode fazer: prometer aprovação, liberar avanço sem política, esquecer coleta crítica, contradizer fato confirmado sem reconciliação.

Pacote mínimo de testes

- Bateria de prompts adversariais para ver se o contrato segura casos de desvio, manipulação e pedido fora de política.
- Teste de consistência entre 20–30 casos repetidos para medir estabilidade de saída estruturada.
- Revisão manual de aderência entre contrato e regras históricas da Enova.

Regra de rollback da frente

Se o contrato ainda permitir resposta bonita mas operacionalmente frouxa, T1 não fecha. O risco aqui é sair com “chat bom” e governança ruim.

T2 — Estado estruturado, memória e reconciliação

Janela-base: Semanas 4–5. Objetivo: Trocar dependência de contexto cru por estado auditável, versionado e reutilizável.

Entregáveis da frente

- Schema lead_state v1 com fatos, objetivos, pendências, conflitos, histórico resumido e metadados de confiança.
- Camada de reconciliação para conflito de fato e atualização incremental.
- Política de confirmação para fatos sensíveis e para mudanças tardias do cliente.
- Compatibilidade transitória com legado enquanto o cutover não chega.

Microetapas obrigatórias

1. Definir nomes canônicos dos fatos e evitar duplicidade semântica entre campos próximos.
2. Separar fato bruto, fato confirmado, inferência, hipótese e pendência.
3. Criar política de confiança por origem: texto explícito, resposta indireta, áudio transcrito, documento lido, inferência semântica.

4. Desenhar o resumo persistido para turnos longos, evitando replay gigante de conversa.
5. Mapear quais fatos continuam vindo do legado até a migração completa e como serão reconciliados.

Pacote mínimo de testes

- Casos de mudança de versão do mesmo fato: renda ajustada, estado civil corrigido, parceiro entra depois, autônomo revela IR depois.
- Teste de conflitos: fato antigo vs fala nova; dado de doc vs fala; inferência vs confirmação explícita.
- Verificação de auditoria: cada update precisa registrar origem, motivo e timestamp lógico.

Regra de rollback da frente

Qualquer perda de rastreabilidade de fato, sobrescrita cega ou conflito silencioso obriga rollback da frente.

T3 — Policy engine v1 e guardrails declarativos

Janela-base: Semanas 6–7. Objetivo: Prender a governança em regras explícitas e testáveis, sem voltar ao trilho engessado.

Entregáveis da frente

- Motor declarativo de políticas com classes: obrigação, bloqueio, sugestão mandatória, confirmação e roteamento.
- Validador pós-resposta e pré-persistência.
- Registro por turno das políticas avaliadas, acionadas, cumpridas e vetadas.
- Biblioteca inicial das regras críticas da Enova.

Microetapas obrigatórias

1. Transformar as regras mais sensíveis primeiro: casado civil → conjunto; autônomo → perguntar IR; renda solo baixa → sugerir composição; estrangeiro sem RNM válido → não avançar.
2. Definir o que é “bloquear avanço”, o que é “desviar objetivo”, o que é “pedir confirmação” e o que é “apenas orientar”.
3. Criar uma ordem estável de avaliação para evitar colisão de políticas.
4. Definir política de composição quando várias regras disparam no mesmo turno.
5. Documentar política de veto suave: o LLM pode falar natural, mas a próxima ação e o update de estado devem obedecer o motor.

Pacote mínimo de testes

- Suíte de regras críticas com casos positivos, negativos e ambíguos.
- Teste de colisão entre regras: ex. autônomo + renda baixa + união estável + composição potencial.
- Teste de regressão para garantir que políticas novas não quebram políticas já aprovadas.

Regra de rollback da frente

Se houver política crítica sem teste ou se o motor não explicar por que vetou/liberou, a frente fica aberta.

T4 — Orquestrador de turno LLM-first

Janela-base: Semanas 8–9. Objetivo: Subir o ciclo único de entrada, interpretação, decisão governada, resposta e persistência.

Entregáveis da frente

- Pipeline padrão para qualquer turno e qualquer canal.
- Saída estruturada única consumível por UI, WhatsApp, voz e telemetria.
- Fallback de segurança para erro do modelo, timeout ou violação de formato.
- Feature flags para shadow mode e cutover parcial.

Microetapas obrigatórias

1. Padronizar a entrada: mensagem, anexos, canal, contexto resumido, estado atual, políticas relevantes e objetivo corrente.
2. Executar o LLM com contrato único e capturar tanto o texto quanto a estrutura.
3. Passar a saída pelo policy engine e pelo reconciliador de estado antes de persistir.
4. Gerar resposta final, registrar o rastro do turno e publicar métricas mínimas.
5. Criar tratamento explícito para erro de modelo, formato inválido, omissão de campos e contradição séria.

Pacote mínimo de testes

- Testes de ponta a ponta em sandbox: entrada → saída → persistência → replay do estado.
- Casos de timeout, resposta inválida, falta de facts obrigatórios e conflito com policy engine.
- Medição de latência e custo por turno para dimensionar operação real.

Regra de rollback da frente

Se o turno não for reproduzível o suficiente para auditoria ou se não houver degradação segura em falha, T4 não fecha.

T5 — Migração do funil prioritário

Janela-base: Semanas 10–12. Objetivo: Levar o novo motor para os fluxos que mais sustentam a operação: topo, identificação, composição, renda, elegibilidade e pré-docs.

Entregáveis da frente

- Mapa de objetivos por macrofase, substituindo dependência direta de stages rígidos.
- Cobertura funcional dos principais perfis da Enova dentro do novo orquestrador.
- Roteamento para docs com política de prontidão, não com escada mecânica cega.
- Comparativo de paridade entre legado e novo motor.

Microetapas obrigatórias

1. Quebrar a migração em fatias: topo e abertura; identificação base; composição/estado civil; renda e regime; elegibilidade e sinais de inviabilidade; envio_docs.

2. Para cada fatia, definir fatos mínimos, políticas mínimas, respostas proibidas e critérios de pronto.
3. Rodar shadow mode por fatia antes de ligar em produção real daquela macrofase.
4. Revisar divergências entre legado e novo motor, classificando em melhoria, regressão, mudança aceitável ou bug.
5. Só desligar dependência do trilho antigo quando a fatia nova superar o mínimo contratual de cobertura e previsibilidade.

Pacote mínimo de testes

- Bateria de casos reais e sintéticos por perfil: solteiro, casado civil, união estável, composição familiar, autônomo com/sem IR, estrangeiro, dependente.
- Comparativo lado a lado de recomendação, fatos extraídos, objetivo escolhido e saída final.
- Métricas mínimas de paridade: coleta correta de fatos críticos, não violação de política e qualidade de condução.

Regra de rollback da frente

Regressão em classificação de perfil, elegibilidade ou encaminhamento de docs obriga rollback da fatia afetada.

T6 — Docs, multimodal e superfícies de canal

Janela-base: Semanas 13–14. Objetivo: Acoplar interpretação de áudio, imagem, sticker e documentos ao mesmo eixo de estado e políticas.

Entregáveis da frente

- Camada de entrada multimodal normalizada.
- Leitura de docs guiada por taxonomia documental da Enova.
- Resposta em texto e áudio dentro das mesmas políticas.
- Telemetria cruzada de canal, latência e acerto de interpretação.

Microetapas obrigatórias

1. Definir representação unificada para texto, transcrição, imagem interpretada, sticker e doc reconhecido.
2. Separar fato extraído do cliente de fato extraído de documento para não misturar confiança.
3. Definir em quais cenários o canal exige confirmação humana ou recapitulação antes de persistir fato crítico.
4. Medir impacto de latência do multimodal e criar fila/estratégia de degradação quando necessário.
5. Garantir que docs não escapem da política de status, pendência e completude do processo.

Pacote mínimo de testes

- Casos de áudio curto, áudio prolixo, imagem irrelevante, imagem útil, sticker sem texto e sticker contextual.
- Casos de doc parcial, doc irrelevante, doc útil porém ambíguo e doc suficiente para atualizar fato.

- Teste de custo/tempo por canal para não explodir operação.

Regra de rollback da frente

Se multimodal reduzir a previsibilidade da coleta crítica ou degradar latência além do tolerável, ele entra por canary mais longo.

T7 — Shadow mode, canary, cutover e desativação do legado

Janela-base: Semanas 15–16 (ou buffer). Objetivo: Trocar efetivamente o eixo do atendimento mantendo rollback imediato e prova de estabilidade.

Entregáveis da frente

- Shadow mode fechado com métricas e análise de divergência.
- Canary progressivo em produção com observação por recorte de fluxo/canal.
- Plano de desligamento das peças legadas substituídas.
- Gate executivo de go-live e runbook de rollback rápido.

Microetapas obrigatórias

1. Rodar o novo motor em paralelo ao legado em amostra representativa sem impactar cliente por uma janela mínima.
2. Classificar divergências e fechar ações antes do canary.
3. Abrir canary por porcentagem, por canal ou por fatia de funil, nunca em 100% de primeira.
4. Medir continuamente acerto operacional, políticas violadas, latência, custo, retrabalho e incidentes.
5. Desligar legado em ordem, com mapa de dependências e prova de que a nova camada cobre o caso.

Pacote mínimo de testes

- Checklist go/no-go com evidências de cobertura e incidentes zerados ou controlados.
- Teste de rollback em ambiente controlado antes do go-live final.
- War room curto após o corte para observar regressão nas primeiras horas e dias.

Regra de rollback da frente

Se a taxa de divergência relevante ou incidente operacional passar do limite definido no contrato, o canary regride ou o tráfego volta ao legado.

4. Critérios de pronto por fase

Fase	Condição mínima de pronto
T0	Mapa do legado aprovado, riscos classificados e dependências de desligamento identificadas.
T1	Contrato cognitivo validado, formato de saída estável e políticas proibitivas documentadas.

T2	Estado versionado persistindo fatos com origem, confiança e conflito auditável.
T3	Regras críticas da Enova codificadas em política declarativa com suíte mínima de testes.
T4	Ciclo de turno completo funcionando com degradação segura, observabilidade e custo medido.
T5	Fluxos prioritários com paridade funcional suficiente e divergências classificadas.
T6	Multimodal sob a mesma governança sem explodir latência nem quebrar previsibilidade.
T7	Shadow mode, canary, rollback e desativação controlada do legado concluídos.

5. Pacote transversal de testes e evidências

- Toda frente deve ter testes sintéticos + recorte de casos reais + análise manual dos casos críticos.
- Toda frente deve publicar evidência mínima: logs, métricas, snapshot de estado, políticas avaliadas e diff contra comportamento esperado quando aplicável.
- Toda regra crítica precisa de pelo menos um caso positivo, um negativo e um ambíguo.
- Toda ativação em produção precisa de smoke tests anteriores, feature flag identificável e comando/caminho claro de rollback.
- Toda mudança que mexa com elegibilidade, docs ou roteamento comercial exige validação reforçada antes de avançar.

6. Organização do trabalho por squads/frentes

Frente	Papel dominante	Responsabilidade	Saída principal
Arquitetura/contrato	Cérebro do projeto	Constituição do agente, políticas, taxonomia, ordem de execução	Documentos canônicos + decisões de desenho
Estado/persistência	Back-end	Schema, reconciliação, compatibilidade e observabilidade	Lead state v1 e trilha auditável

Policy/orquestração	Back-end + IA	Motor declarativo, veto, turno unificado e feature flags	Policy engine + orquestrador LLM-first
QA/telemetria	QA + operação	Baterias, shadow mode, canary, métricas e incidentes	Relatórios de evidência e go/no-go
Canais/multimodal	Integração	Áudio, imagem, sticker, docs e superfícies de saída	Camada multicanal governada

7. Ordem dos próximos documentos

Próximo artefato	Função
PDF 4	Backlog Mestre — épicos, histórias, prioridades, dependências e critérios de aceite.
PDF 5	Playbook Operacional — cadência semanal, rituais, incidentes, shadow mode, canary e rollback.
PDF 6	Taxonomia Canônica — facts, objetivos, políticas, eventos, riscos, canais e estados do novo modelo.
PDF 7	Contrato Rígido de Ordem de Execução — sequência inviolável, gates, bloqueios de escopo e regras de mudança.

Observação executiva final: o plano tático não substitui backlog, playbook, taxonomia ou contrato rígido. Ele diz o que fazer e em que ordem; os próximos documentos vão destrinchar a fila de trabalho, a operação do dia a dia e a ontologia oficial do novo motor.

BACKLOG MESTRE

PDF 4 de 7 — Inventário executivo, priorização, dependências, critérios de aceite e ordem real de construção do novo modelo Enova LLM-first, substituindo o motor mecânico e o motor cognitivo atual com governança progressiva.

Função desta peça

Transformar o plano e o contrato em backlog operacional rastreável: épicos, pacotes de trabalho, critérios de pronto, estimativas, dependências, bloqueios, sequencing, gates e política de execução para evitar drift, retrabalho e mistura indevida de frentes.

Status	Canônico v1.0
Natureza	Backlog mestre executivo + técnico
Leituras prévias	PDF 1 — Plano Canônico Macro; PDF 2 — Contrato de Implantação; PDF 3 — Plano Tático de Execução
Horizonte-base	15 semanas + 2 semanas de buffer controlado
Unidade de gestão	Épico → pacote → microtarefa → evidência → gate
Regra obrigatória	Nada entra em execução sem depender explicitamente da ordem e dos gates definidos neste documento

Regra executiva central: o backlog não é lista solta de tarefas. Ele é a ordem canônica da substituição dos motores. Tarefas fora de ordem, misturadas ou sem cobertura de aceite são consideradas execução inválida mesmo que “funcionem” localmente.

1. Finalidade do backlog mestre

Este backlog existe para impedir improviso. Como a mudança proposta substitui o eixo de decisão da Enova, o backlog precisa funcionar como mapa de guerra: o que vem antes, o que não pode ser misturado, quais blocos são estruturais, quais entregas só começam após prova de prontidão e quais cortes devem ser feitos apenas depois de cobertura comprovada do modelo novo.

- Separar backlog estrutural de backlog de migração de fluxo e backlog multimodal.
- Amarrar cada pacote a um dono, estimativa, evidência mínima e rollback.
- Priorizar redução de risco antes de ganho cosmético.
- Proibir execução oportunista fora da ordem rígida definida neste documento.

2. Legenda de prioridade e criticidade

Código	Significado operacional
P0	Sem isso o novo modelo não pode existir ou não pode entrar em sombra/produção.
P1	Necessário para cobrir o core do funil e viabilizar migração com segurança.
P2	Amplia cobertura, canais e governança, mas não deve bloquear o core.
P3	Otimização, ergonomia, reporting avançado, ganhos pós-estabilização.
C1 — Crítico	Falha derruba elegibilidade, decisão, roteamento, persistência ou rollback.
C2 — Alto	Falha afeta qualidade operacional, cobertura ou confiança de uso.
C3 — Médio	Falha incomoda, reduz eficiência ou observabilidade, mas não inviabiliza o core.

3. Ondas oficiais de execução

A implantação foi quebrada em ondas para impedir o erro clássico de construir tudo ao mesmo tempo. Cada onda tem objetivo próprio, pacote de saídas e condição de abertura da próxima.

Onda	Janela	Escopo obrigatório

Onda 0	Semanas 1–2	Congelamento do legado, inventário vivo, taxonomia, contrato do agente e critérios de cobertura.
Onda 1	Semanas 3–5	Estado estruturado, memória operacional, reconciliação e policy engine P0.
Onda 2	Semanas 6–8	Orquestrador de turno, saída estruturada, telemetria fina, harness de avaliação.
Onda 3	Semanas 9–11	Migração do topo ao bloco de elegibilidade/renda com shadow mode e paridade assistida.
Onda 4	Semanas 12–13	Docs, multimodal e hardening de canais.
Onda 5	Semanas 14–15	Canary, cutover progressivo, desligamento controlado do legado e estabilização.

4. Backlog mestre por épico

E0 — Congelar legado e fechar mapa real de dependências

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semana 1 | Dono primário: Arquitetura + Diagnóstico

Dependências de abertura: Nenhuma. É o ponto zero obrigatório.

Descobrir o que está vivo hoje no motor mecânico, no cognitivo atual, na persistência, na telemetria, nos canais e nos fluxos de docs. Sem essa base, qualquer substituição vira chute.

E0.1 Inventário do legado vivo

Estimativa do pacote: 2 dias

- Escopo: Mapear stages, gates, regras críticas, side effects, dependências de tabelas, flags, parsers e endpoints realmente usados.
- Critério de aceite: Inventário assinado e sem “ilhas cinzentas” críticas.
- Evidência mínima: Documento de inventário + matriz legado→novo.
- Não iniciar se: houver trecho do legado sem owner e sem classificação de risco.

E0.2 Mapa de corte e convivência

Estimativa do pacote: 1 dia

- Escopo: Definir o que será mantido temporariamente, o que entra em sombra e o que será desligado apenas no cutover.
- Critério de aceite: Mapa de convivência por componente aprovado.
- Evidência mínima: Quadro de coexistência com feature flags e rollback.
- Não iniciar se: não existir plano explícito de rollback por componente.

E0.3 Linha de base de testes

Estimativa do pacote: 2 dias

- Escopo: Congelar suíte atual de casos reais e sintéticos para usar como benchmark do novo modelo.
- Critério de aceite: Suíte-base versionada e reproduzível.
- Evidência mínima: Lista de casos + runner + outputs esperados.
- Não iniciar se: não houver casos cobrindo renda, autônomo, estado civil, RNM, restrição e docs.

E1 — Constituição do agente e taxonomia canônica

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semanas 1–2 | Dono primário: Produto Cognitivo + Arquitetura

Dependências de abertura: E0 concluído.

Transformar o conhecimento do funil em contrato executável: identidade do agente, objetivos por fase, políticas duras, fatos do lead, saídas estruturadas, limites de linguagem e protocolos de confirmação.

E1.1 System prompt constitucional

Estimativa do pacote: 2 dias

- Escopo: Escrever a constituição do agente LLM-first com papel, objetivos, limites, comportamento em dúvida, bloqueios, tom e formato de saída.
- Critério de aceite: Prompt cobre identidade, proibições, fallback, confirmação e objetivos.
- Evidência mínima: Prompt versionado + checklist de cobertura.
- Não iniciar se: ainda faltar regra dura do negócio sem representação explícita.

E1.2 Taxonomia de fatos e slots

Estimativa do pacote: 2 dias

- Escopo: Definir fatos canônicos: estado civil, processo, composição, regimes, IR, CTPS, restrição, nacionalidade, RNM, docs, canal e metas de coleta.
- Critério de aceite: Taxonomia cobre todo o core do funil.
- Evidência mínima: Dicionário de fatos com tipos, fonte, confiança e conflitos.
- Não iniciar se: campos críticos não tiverem definição única.

E1.3 Catálogo de objetivos por fase

Estimativa do pacote: 2 dias

- Escopo: Converter o funil em objetivos semânticos: o que descobrir, o que confirmar, quando bloquear, quando sugerir, quando encaminhar.

- Critério de aceite: Cada fase tem objetivo, saída, bloqueios e sinais aceitos.
- Evidência mínima: Matriz objetivo→entrada→saída.
- Não iniciar se: objetivos dependerem de texto solto sem critério de conclusão.

E2 — Estado estruturado e memória operacional do lead

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semanas 3–4 | Dono primário: Engenharia de Dados + Backend

Dependências de abertura: E1 concluído.

Criar a representação persistida do caso para que o modelo converse livre, mas opere sobre fatos, pendências, conflitos, histórico resumido e próximos objetivos válidos.

E2.1 Schema do lead_state v1

Estimativa do pacote: 2 dias

- Escopo: Definir objeto persistido com facts, confidence, pending_questions, risk_flags, policy_status, channel_state e audit trail.
- Critério de aceite: Schema cobre core do funil e suporta expansão.
- Evidência mínima: Contrato JSON/DB + migração aplicada em TEST.
- Não iniciar se: o schema depender de campo legado obscuro sem justificativa.

E2.2 Estratégia de memória curta e longa

Estimativa do pacote: 1 dia

- Escopo: Separar resumo operacional do histórico bruto, evitando contexto inflado e inconsistência.
- Critério de aceite: Regras de compactação e resumo aprovadas.
- Evidência mínima: Documento de memória + exemplos.
- Não iniciar se: não houver regra de reconciliação em caso de resposta contraditória.

E2.3 Reconciliação e versionamento de fatos

Estimativa do pacote: 2 dias

- Escopo: Implementar atualização segura de fatos com confiança, sobrescrita, revisão e trilha de quem mudou o quê.
- Critério de aceite: Conflitos detectados e auditáveis.
- Evidência mínima: Testes de conflito e merge de fatos.
- Não iniciar se: updates sobrescreverem fatos críticos sem prova ou confirmação.

E3 — Policy engine declarativo

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semanas 4–5 | Dono primário: Arquitetura + Backend

Dependências de abertura: E2 concluído.

Tirar as regras do escuro e convertê-las em políticas declarativas, auditáveis e expansíveis, sem cair no mecânico pesado anterior.

E3.1 Biblioteca de políticas P0

Estimativa do pacote: 2 dias

- Escopo: Codificar regras como autônomo→perguntar IR, casado civil→processo conjunto, estrangeiro sem RNM válido→não avançar, renda solo baixa→sugerir composição.
- Critério de aceite: Todas as políticas P0 passam em suíte dedicada.
- Evidência mínima: Matriz regra→teste→resultado.
- Não iniciar se: qualquer política P0 depender de interpretação implícita do modelo.

E3.2 Engine de obrigação/bloqueio/sugestão

Estimativa do pacote: 1 dia

- Escopo: Criar classes de regra e decisão: required_ask, route, block, confirm, recommend.
- Critério de aceite: Engine suporta expansão por configuração.
- Evidência mínima: Documento do DSL/config + testes.
- Não iniciar se: a inclusão de regra nova exigir reescrita do orquestrador.

E3.3 Explainability mínima

Estimativa do pacote: 1 dia

- Escopo: Registrar qual política rodou e por que uma pergunta/rota foi obrigada, vetada ou sugerida.
- Critério de aceite: Cada turno crítico deixa trilha clara.
- Evidência mínima: Log estruturado policy_trace.
- Não iniciar se: a auditoria não conseguir explicar o motivo do passo seguinte.

E4 — Orquestrador de turno LLM-first

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semanas 6–7 | Dono primário: Backend + IA Aplicada

Dependências de abertura: E2 e E3 concluídos.

Unificar o ciclo do turno: entrada do canal, interpretação LLM, extração de fatos, checagem de políticas, decisão do próximo objetivo, resposta, persistência e telemetria.

E4.1 Contrato de entrada e saída

Estimativa do pacote: 1 dia

- Escopo: Padronizar input por canal e output estruturado do modelo: reply_text, facts, pending, next_goal, needs_confirmation, policy_checks.
- Critério de aceite: Contrato aceito por todos os canais-alvo.
- Evidência mínima: Schema v1 + exemplos válidos.
- Não iniciar se: canais precisarem de formatos ad hoc diferentes sem adaptador claro.

E4.2 Loop de decisão do turno

Estimativa do pacote: 2 dias

- Escopo: Implementar pipeline único do turno sem lógicas escondidas em ilhas.

- Critério de aceite: Turno executa do início ao fim com idempotência básica.
- Evidência mínima: Fluxo rodando em TEST com logs completos.
- Não iniciar se: persistência ou telemetria ainda estiverem acopladas a trechos legados obscuros.

E4.3 Recovery e retry seguro

Estimativa do pacote: 1 dia

- Escopo: Definir comportamento em timeout, saída inválida, canal falho, ambiguidade ou quebra de contrato.
- Critério de aceite: Erros críticos têm fallback controlado.
- Evidência mínima: Tabela de recovery + testes.
- Não iniciar se: falha de um componente levar a resposta sem governança.

E5 — Telemetria, avaliação e harness de regressão

Prioridade: P0 | Criticidade: C2 | Estimativa-base: Semanas 6–8 | Dono primário: Observabilidade + QA

Dependências de abertura: E4 aberto; pode iniciar em paralelo controlado.

Sem avaliação e telemetria fina, o novo modelo vira opinião. Esta frente mede cobertura, deriva, violação de regra, latência e qualidade real de cada turno.

E5.1 Telemetria de turno novo

Estimativa do pacote: 2 dias

- Escopo: Emitir eventos do novo pipeline: entrada, facts_detected, policy_applied, next_goal, reply_generated, confirmation_needed, final_outcome.
- Critério de aceite: Eventos aparecem por lead e por execução.
- Evidência mínima: Endpoints/consultas entregues e validados.
- Não iniciar se: não existir correlação por lead/turno.

E5.2 Harness de casos reais

Estimativa do pacote: 2 dias

- Escopo: Runner com casos do funil e outputs esperados por política e objetivo.
- Critério de aceite: Suite cobre pelo menos 80% dos casos P0/P1.
- Evidência mínima: Relatório comparando legado x novo.
- Não iniciar se: cobertura não incluir casos cinzentos reais.

E5.3 Scorecard executivo

Estimativa do pacote: 1 dia

- Escopo: Definir métricas de go/no-go: cobertura, policy pass rate, confirmação correta, latência, queda de resposta, regressão comercial.
- Critério de aceite: Painel de decisão pronto.
- Evidência mínima: Planilha/painel com thresholds assinados.

- Não iniciar se: não houver threshold oficial para abrir sombra ou canary.

E6 — Migração do funil core

Prioridade: P1 | Criticidade: C1 | Estimativa-base: Semanas 8–11 | Dono primário: Produto + Backend + QA

Dependências de abertura: E4 e E5 com estabilidade mínima.

Migrar o coração da Enova do topo até elegibilidade, mantendo governança e paridade com as regras de negócio mais críticas.

E6.1 Topo e apresentação

Estimativa do pacote: 2 dias

- Escopo: Entrada, acolhimento, explicação do programa, qualificação inicial e retorno ao objetivo.
- Critério de aceite: Topo natural sem perder captação do primeiro sinal útil.
- Evidência mínima: Casos de curiosidade, desvio e abertura aprovados.
- Não iniciar se: não houver política clara de retorno ao objetivo.

E6.2 Estado civil, composição e roteamento

Estimativa do pacote: 3 dias

- Escopo: Regras de solo, conjunto, união estável, casado civil, composição familiar e P3.
- Critério de aceite: Roteamento cobre todos os cenários P0/P1 conhecidos.
- Evidência mínima: Matriz de casos validada.
- Não iniciar se: casos de composição ainda dependerem de heurística obscura.

E6.3 Regime, renda, IR, CTPS, dependente, restrição e RNM

Estimativa do pacote: 4 dias

- Escopo: Migrar regras centrais de elegibilidade e coleta crítica.
- Critério de aceite: Policy pass rate acima do threshold em suíte real.
- Evidência mínima: Relatório de paridade por regra.
- Não iniciar se: qualquer regra crítica ainda não tiver caso de teste explícito.

E7 — Docs, multimodal e canais

Prioridade: P1 | Criticidade: C2 | Estimativa-base: Semanas 11–13 | Dono primário: Canais + IA Aplicada + Produto

Dependências de abertura: E6 com paridade mínima aprovada.

Acoplar leitura e resposta multimodal sem quebrar governança, trazendo áudio, imagem, sticker e docs para o mesmo contrato operacional.

E7.1 Adaptadores de canal

Estimativa do pacote: 2 dias

- Escopo: Padronizar texto, áudio, imagem e sticker para o contrato de entrada do orquestrador.

- Critério de aceite: Adaptadores entregam payload único e consistente.
- Evidência mínima: Testes multimodais por canal.
- Não iniciar se: um canal exigir atalho fora do orquestrador.

E7.2 Políticas multimodais

Estimativa do pacote: 1 dia

- Escopo: Definir quando transcrever, quando resumir, quando pedir confirmação e como tratar baixa confiança.
- Critério de aceite: Regras multimodais aprovadas.
- Evidência mínima: Checklist multimodal + casos.
- Não iniciar se: não houver guarda contra baixa confiança em áudio/imagem.

E7.3 Docs no novo modelo

Estimativa do pacote: 2 dias

- Escopo: Conectar dossiê, recebimento, status e handoff no mesmo estado estruturado.
- Critério de aceite: Docs entram no novo eixo sem regressão do que já funciona.
- Evidência mínima: Casos de envio parcial/completo e reconhecimento básico.
- Não iniciar se: status documental perder compatibilidade operacional.

E8 — Shadow, canary, cutover e desligamento

Prioridade: P0 | Criticidade: C1 | Estimativa-base: Semanas 14–15 | Dono primário: Operação + Arquitetura + QA

Dependências de abertura: E6 e E7 aprovados.

Trocar com segurança o motor antigo pelo novo, sem salto cego. Primeiro sombra, depois canary controlado, depois cutover progressivo e por fim desligamento do legado que ficou comprovadamente coberto.

E8.1 Shadow mode oficial

Estimativa do pacote: 2 dias

- Escopo: Executar novo pipeline em paralelo sem responder ao cliente, comparando decisões, fatos e políticas.
- Critério de aceite: Diferenças relevantes classificadas e tratadas.
- Evidência mínima: Relatório shadow por amostra real.
- Não iniciar se: shadow ainda não tiver métricas de divergência e severidade.

E8.2 Canary por fatia

Estimativa do pacote: 2 dias

- Escopo: Ligar novo motor para uma fatia controlada de leads/canais.
- Critério de aceite: Canary fica estável dentro dos thresholds.
- Evidência mínima: Relatório diário de canary.

- Não iniciar se: não existir rollback instantâneo por flag.

E8.3 Cutover e desligamento

Estimativa do pacote: 1 dia

- Escopo: Promover o novo motor, manter fallback temporário e desligar legado coberto.
- Critério de aceite: Go-live sem regressão severa e com rollback pronto.
- Evidência mínima: Ata de go/no-go + plano de desligamento.
- Não iniciar se: alguma dependência crítica do legado ainda não estiver coberta.

E9 — Hardening pós-go-live e expansão de regras

Prioridade: P2 | Criticidade: C2 | Estimativa-base: Buffer semanas 16–17 | Dono primário: Operação + Produto

Dependências de abertura: E8 concluído.

Consolidar o novo modelo, absorver regras ainda fora do funil atual, ampliar reporting, otimizar custo/latência e transformar aprendizados em política declarativa nova.

E9.1 Backlog de expansão

Estimativa do pacote: 2 dias

- Escopo: Trazer regras hoje tácitas para a base declarativa sem reabrir arquitetura.
- Critério de aceite: Novo lote de políticas priorizado.
- Evidência mínima: Lista priorizada P1/P2 futura.
- Não iniciar se: expansão começar alterando orquestrador em vez de política/estado.

E9.2 Performance e custo

Estimativa do pacote: 1 dia

- Escopo: Ajustar prompts, memória e roteamento para reduzir latência e custo.
- Critério de aceite: Meta mínima de latência definida e atacada.
- Evidência mínima: Medição antes/depois.
- Não iniciar se: otimização reduzir cobertura ou governança.

E9.3 Manual operacional v1

Estimativa do pacote: 1 dia

- Escopo: Fechar SOP de operação, incidentes, rollout e evolução do novo motor.
- Critério de aceite: Manual entregue e utilizável.
- Evidência mínima: Documento operacional final.
- Não iniciar se: operação ainda depender de conhecimento oral não documentado.

5. Matriz rígida de dependências e bloqueios

Regra	Condição

E1 só abre após	E0 completo e linha de base de testes congelada
E2 só abre após	E1 com taxonomia e contrato fechados
E3 só abre após	E2 com schema e reconciliação definidos
E4 só abre após	E2 + E3 estáveis em TEST
E5 pode paralelizar com	E4, desde que use contrato já congelado
E6 só abre após	E4 funcional + E5 com scorecard mínimo
E7 só abre após	E6 com paridade mínima aprovada
E8 só abre após	E6 + E7 com suite e telemetria dentro do threshold
E9 só abre após	E8 concluído e go-live estabilizado

Misturas proibidas

- Não misturar construção de política com improviso de prompt em produção sem atualizar a taxonomia e a suíte de casos.
- Não misturar cutover com criação de canal novo.
- Não desligar legado por “sensação de que está bom”; desligamento só com evidência shadow/canary.
- Não abrir expansão P2/P3 antes de estabilizar P0/P1.

6. Critérios de pronto por classe de item

Classe	Definição de pronto
Contrato	Versionado, revisado, sem conflito semântico com regras existentes e com owners definidos.
Schema/estado	Persistido em TEST, com exemplos válidos, migração reversível e testes de conflito.
Política	Existe em configuração/código declarativo, tem caso positivo e negativo, e deixa trace auditável.
Orquestrador	Ciclo do turno completo, idempotência mínima,

	telemetria emitida e recovery definido.
Migração de fluxo	Paridade comprovada no scorecard oficial e sem regressão crítica nos casos reais.
Canal multimodal	Adaptador respeita contrato único, baixa confiança pede confirmação e logs funcionam.
Cutover	Shadow aprovado, canary aprovado, rollback pronto e decisão formal de go/no-go registrada.

7. Ordem oficial dos próximos documentos

Próxima peça	Nome	Função
PDF 5	Playbook de Implantação e Operação	Como executar semana a semana, abrir frentes, registrar evidências, conduzir incidentes e decidir go/no-go.
PDF 6	Taxonomia, Estado Estruturado e Policy Model	Manual formal dos fatos, objetos, políticas, classes de regra e contrato do estado do lead.
PDF 7	Ordem Rígida de Execução + Macro e Microplanejamento Final	Sequência definitiva, calendário fechado, microetapas por semana, critérios executivos e trilho obrigatório de substituição dos motores.

PLAYBOOK DE IMPLANTAÇÃO E OPERAÇÃO

PDF 5 de 7 — Rotina executiva, ceremonies, controle de mudança, playbooks de incidente, shadow/canary/cutover e desenho operacional de persistência no Supabase para o novo modelo Enova LLM-first.

Função desta peça

Transformar o plano, o contrato, o tático e o backlog em rotina de execução disciplinada. Este documento define como o time trabalha no dia a dia, como registra fatos e decisões no Supabase, como sobe shadow/canary, como reage a incidente e como promove cortes sem cair em drift ou perda de rastreabilidade.

Status	Canônico v1.0
Natureza	Playbook executivo-operacional + persistência de dados
Leituras prévias	PDF 1 — Plano Canônico Macro; PDF 2 — Contrato de Implantação; PDF 3 — Plano Tático; PDF 4 — Backlog Mestre
Horizonte-base	Semanas 1 a 15 + estabilização contínua
Escopo	Implantação, operação, governança, incidente, rollback e base de persistência
Regra obrigatória	Nenhuma mudança em prompt, policy, estado ou rollout acontece fora dos ritos definidos neste playbook

Resposta objetiva à dúvida crítica: sim, o novo modelo precisa de persistência estruturada e o Supabase continua sendo a espinha dorsal recomendada para estado do lead, memória factual, logs por turno, decisões de policy, catálogo de prompts, datasets de avaliação e trilha de rollout. O LLM conversa; o Supabase guarda a verdade operacional.

1. Finalidade operacional do playbook

A troca dos motores da Enova não falha por falta de ideia; ela falha se a execução virar improviso. Este playbook existe para tornar a implantação repetível, auditável e disciplinada. Ele vale tanto para o time técnico quanto para o decisor. Qualquer execução fora dele aumenta risco de drift, regressão e falsa sensação de avanço.

- Padronizar ritos semanais, gates de mudança e rotina de decisão.
- Definir como o estado do lead e os eventos do agente são persistidos no Supabase.
- Dar protocolo de ação para shadow, canary, cutover, rollback e incidente.
- Proibir ajuste improvisado em produção sem lastro em evidência e rollback.

2. Princípio central de operação

Fórmula canônica: LLM livre na conversa, estado estruturado no banco, policy governando decisão, telemetria registrando tudo e rollout sempre progressivo.

3. Cadência oficial de trabalho

Rito	Frequência	Objetivo
Daily de implantação	20 min	Bloqueios, progresso real, risco e próximo passo fechado.
Review de evidências	3x por semana	Verificar datasets, telemetria, violações de policy e regressões.
Change review	2x por semana	Aprovar ou reprovar mudanças em prompt, policy, schema e rollout.
Checkpoint executivo	1x por semana	Comparar cronograma, risco, decisão de continuidade e custos.
Postmortem de incidente	até 24h após incidente	Fechar causa, impacto, contenção e prevenção.

4. Artefatos mínimos obrigatórios por mudança

- hipótese da mudança
- componente afetado (prompt, policy, estado, telemetria, canal, docs, rollout)
- evidência que motivou a mudança

- teste antes/depois
- risco estimado
- plano de rollback
- critério de aceite

5. Supabase como espinha dorsal do novo modelo

Sim: o novo modelo deve persistir estado e memória no Supabase. O erro seria tentar operar um agente sofisticado só com contexto de prompt. O banco passa a guardar a verdade operacional do caso, e não apenas restos do legado. Isso permite reentrada, auditoria, reconciliação, canary e debugging real.

5.1 Objetos de persistência recomendados

Tabela	Função	Campos-núcleo
enova_leads	Identidade operacional do lead	lead_id, wa_id, canal, origem, status, timestamps
enova_sessions	Sessão conversacional/atendimento	session_id, lead_id, started_at, ended_at, channel_mode
enova_state_v2	Estado estruturado atual	lead_id, current_phase, current_objective, process_mode, completion_pct, risk_level
enova_facts	Fatos atômicos confirmados ou pendentes	fact_id, lead_id, fact_key, fact_value, confidence, source_turn_id, status
enova_turns	Log por turno	turn_id, lead_id, role, input_type, raw_input, reply_text, model_name, latency_ms
enova_policy_events	Decisões e travas de policy	turn_id, rule_code, action_type, outcome, reason, severity
enova_memory_snapshots	Resumo de memória operacional	lead_id, summary_text, open_questions, contradictions, next_must_ask
enova_prompt_registry	Versões do contrato do agente	prompt_version, scope, hash, status, activated_at

enova_eval_runs	Resultados de avaliação offline/online	run_id, dataset_version, score, regressions, created_at
enova_rollout_events	Shadow/canary/cutover/rollback	event_id, env, cohort, action, trigger, evidence_link
enova_artifacts	Áudios, imagens, docs, transcripts e derivados	artifact_id, lead_id, type, storage_path, parsing_status

5.2 Regras de escrita na base

- cada turno do agente grava uma linha em enova_turns, com input original, output final, latência, modelo e referência de sessão;
- todo fato extraído precisa ir para enova_facts com status claro: inferred, confirmed, contradicted, superseded ou discarded;
- o estado consolidado do lead mora em enova_state_v2 e não deve ser reconstruído por leitura oportunista de log bruto;
- qualquer trava ou obrigação de policy deve gerar enova_policy_events;
- resumos de memória devem ser compactos e operacionais; nunca depender só do histórico inteiro para retomar um caso;
- mudança de prompt ativa deve ser versionada em enova_prompt_registry e referenciada pelos turnos gravados;

5.3 Estratégia de memória

A memória não deve ser um bloco textual solto. Ela deve ser híbrida: fatos estruturados + snapshot resumido. Exemplo: o lead pode já ter informado “sou autônomo e ganho 2.400”, mas a memória operacional precisa guardar isso como fato estruturado e, ao mesmo tempo, registrar no snapshot que ainda falta confirmar IR e que existe alta probabilidade de sugerir composição.

Camada	Conteúdo	Uso
Fatos	verdades atômicas do caso	regras, policy, roteamento e busca precisa
Snapshot	síntese do estado atual	retomada rápida do atendimento
Turns	histórico bruto	auditoria, debugging e reprocessamento

6. Playbooks operacionais obrigatórios

6.1 Novo fato ou nova regra de negócio

- abrir diagnóstico com evidência real do caso ou da regra;
- classificar se é regra de obrigação, bloqueio, sugestão obrigatória ou confirmação;
- adicionar o slot/fato na taxonomia e no schema de enova_facts ou enova_state_v2 se necessário;
- criar/ajustar rule_code no policy engine;
- rodar casos de avaliação afetados;
- só depois promover a mudança para shadow/canary.

6.2 Mudança no system prompt constitucional

- toda alteração precisa de hipótese clara e dataset de regressão;
- versionar novo prompt em enova_prompt_registry;
- ativar primeiro em shadow, nunca direto em canary amplo;
- comparar taxa de violação, latência, completude e aderência ao objetivo;
- manter caminho de rollback para a versão anterior.

6.3 Entrada de novo canal multimodal

- separar entendimento do conteúdo bruto da decisão do funil;
- registrar artefato em enova_artifacts;
- gerar transcript/descrição derivada e linkar ao turn;
- extrair fatos com confiança explícita;
- se baixa confiança, confirmar com o cliente antes de avançar.

6.4 Shadow mode

- o novo agente roda em paralelo ao fluxo ativo sem impactar o cliente;
- comparar fatos extraídos, decisões de policy e próximo objetivo;
- medir divergência por classe de caso;
- abrir correção apenas em cima de divergência comprovada.

6.5 Canary

- subir coorte pequena e observável;
- limitar classes de caso inicialmente;
- acompanhar latência, violação de regra, regressão de coleta e necessidade de intervenção manual;
- qualquer violação crítica abre rollback imediato do cohort.

6.6 Incidente e rollback

- conter primeiro, investigar depois;

- desativar prompt/policy/feature flag ofensora;
- registrar rollout_event e incidente;
- restaurar rota estável;
- fazer postmortem em até 24h com causa, impacto e prevenção.

7. Gates operacionais de promoção

Gate	Condição de abertura
Gate A — Design pronto	taxonomia, prompt, policy e schema aprovados
Gate B — Shadow pronto	divergência dentro do limite e sem violação crítica
Gate C — Canary pronto	coorte controlada estável e sem regressão P0/P1
Gate D — Cutover parcial	tráfego ampliado com monitoramento fino e rollback imediato pronto
Gate E — Cutover amplo	novo modelo cobre casos-alvo com estabilidade sustentada
Gate F — Desligamento legado	somente após janela de segurança e prova de não dependência real

8. Indicadores obrigatórios por semana

- cobertura de casos do dataset canônico
- taxa de violação de policy por 100 conversas
- latência média e p95 por turno
- taxa de confirmação necessária por extração ambígua
- taxa de intervenção manual
- percentual de leads com estado reconciliado sem contradições abertas
- regressões novas versus baseline

9. Macro e microetapas com prazo operacional

Janela	Marco obrigatório
Semana 1	congelar legado, fechar inventário e baseline de

	testes
Semana 2	fechar taxonomia, prompt constitucional e primeiro schema Supabase v2
Semana 3	subir state_v2, facts, turns e policy_events em ambiente de teste
Semana 4	orquestrador de turno + memória snapshot + registry de prompts
Semana 5	harness de avaliação e primeiros datasets de regressão
Semana 6	shadow mode do topo do funil e elegibilidade inicial
Semana 7	shadow de renda, autônomo, IR, estado civil e composição
Semana 8	shadow ampliado com RNM, restrição e CTPS
Semana 9	primeiro canary restrito
Semana 10	ajustes de policy, memória e datasets
Semana 11	canary ampliado e monitorado
Semana 12	docs + artefatos multimodais integrados
Semana 13	hardening de canais e operação
Semana 14	cutover parcial e redução do legado
Semana 15	cutover amplo, estabilização e plano de desligamento

9.1 Microetapas diárias por frente

- diagnóstico e evidência;
- desenho da mudança;
- ajuste de schema/prompt/policy;
- teste sintético;
- teste em dataset real;

- registro de evidências;
- gate de revisão;
- promoção ou rollback.

10. Fechamento executivo

Este playbook responde ao ponto mais sensível da migração: como o novo modelo conversa livremente sem perder governança. A resposta não é voltar ao mecânico duro; é operar com disciplina. O LLM passa a ser o motor de interação. O Supabase passa a guardar a verdade operacional do caso. O policy engine segura regra de negócio. E o rollout progressivo impede salto no escuro.

Próximo documento oficial: PDF 6 — Taxonomia, Modelo de Dados e Política de Estado/Memória.

TAXONOMIA, MODELO DE DADOS E POLÍTICA DE ESTADO/MEMÓRIA

PDF 6 de 7 — Contrato canônico dos fatos, entidades, estruturas persistidas no Supabase, política de atualização de estado, memória híbrida e formato obrigatório de saída do agente Enova LLM-first.

Função desta peça

Este documento define a espinha dorsal do novo modelo. Ele substitui a dependência de stage mecânico rígido por um estado estruturado do lead, facts versionados, memória operacional útil, eventos de policy e contrato de saída do LLM. Sem este documento, o agente fica solto; com ele, a conversa pode ser natural sem perder governança.

Status	Canônico v1.0
Natureza	Taxonomia + schema operacional + política de estado/memória
Leituras prévias	PDF 1 — Plano Macro; PDF 2 — Contrato de Implantação; PDF 3 — Plano Tático; PDF 4 — Backlog; PDF 5 — Playbook
Horizonte-base	Estrutura permanente do modelo
Escopo	Facts, entidades, atualização de estado, memória, consistência e saída estruturada
Regra obrigatória	Nenhuma conversa, rollout ou automação pode escapar desta taxonomia sem change review formal

Princípio-base: o agente conversa livremente, mas persiste fatos, atualiza o estado e avança objetivos apenas dentro de um contrato explícito. A memória não substitui a regra; a memória preserva contexto útil para que a regra seja aplicada com inteligência.

1. O que este documento substitui no legado

No modelo antigo, o stage mecânico carregava três responsabilidades ao mesmo tempo: orientação de conversa, governança de decisão e registro de progresso. Isso gerava rigidez, explosão de edge cases e manutenção cara. No novo modelo, essas responsabilidades são separadas: conversa no LLM, governança no policy engine, persistência no banco e memória em snapshots operacionais.

- O estado do lead substitui o stage como referência principal de progresso.
- Facts substituem parsing solto e memória implícita.
- Policy events substituem heurística escondida na conversa.
- Memory snapshots substituem repetição burra de histórico extenso no prompt.

2. Entidades canônicas do novo modelo

Entidade	Papel	Definição prática
Lead	Entidade raiz do caso	Uma pessoa/contato com identidade operacional.
Session	Janela de atendimento	Conjunto de turnos em um contexto específico.
Turn	Interação atômica	Entrada + resposta + metadados do modelo.
Fact	Dado estruturado do caso	Informação extraída, confirmada, pendente ou contraditória.
State	Visão corrente governada	Resumo objetivo do caso e do próximo objetivo.
Policy Event	Ato de governança	Bloqueio, obrigação, sugestão ou roteamento acionado.
Memory Snapshot	Resumo executivo do caso	Contexto comprimido para reentrada eficiente.
Artifact	Mídia e derivados	Áudio, imagem, documento, transcrição, OCR, embeddings futuros.
Eval Run	Bateria de validação	Métrica e regressão do agente em datasets.

3. Taxonomia oficial de facts

Os facts são o núcleo da disciplina do agente. Cada fact deve ter chave estável, valor, status, fonte, confiança e timestamps. Nada crítico pode ficar apenas “na cabeça do modelo”. A taxonomia abaixo é a base mínima e pode crescer por change control.

F0 — Identidade e contexto base

lead_name: Nome principal do cliente

preferred_name: Como prefere ser chamado

channel_origin: Origem do lead/canal

language_mode: pt-BR, outro idioma, mistura

customer_goal: Comprar imóvel, entender programa, enviar docs, visitar etc.

F1 — Nacionalidade e elegibilidade documental

nationality: Brasileiro, estrangeiro, naturalizado

rnm_required: Se RNM é obrigatório

rnm_status: válido, vencido, ausente, indeterminado

document_identity_type: RG, CNH, identidade nova, passaporte etc.

F2 — Estado civil e modo de processo

marital_status: solteiro, união estável, casado civil, divorciado, viúvo

process_mode: solo, conjunto, composicao_familiar

composition_actor: cônjuge, parceiro, pai, mãe, irmão etc.

p3_required: se precisa terceiro participante

F3 — Trabalho, renda e formalização

work_regime_p1: CLT, autônomo, aposentado, servidor, informal, múltiplo

monthly_income_p1: renda declarada principal

has_multi_income_p1: sinal de múltiplas fontes

autonomo_has_ir_p1: sim, não, parcial, não informado

ctps_36m_p1: sim, não, parcial, não informado

F4 — Co-participantes

work_regime_p2: regime do parceiro/compositor

monthly_income_p2: renda do parceiro/compositor

autonomo_has_ir_p2: IR do parceiro

ctps_36m_p2: CTPS do parceiro

work_regime_p3: regime do terceiro participante

F5 — Restrições, reservas e alavancas

credit_restriction: há restrição?

restriction_regularization_status: regularizada, em andamento, não regularizada

has_fgts: sim/não

entry_reserve_signal: tem ou não reserva para entrada

benefits_signal: bolsa família, aposentadoria, outros sinais úteis

F6 — Dependente e subsídio

dependents_count: número de dependentes quando aplicável

dependents_applicable: se a pergunta faz sentido para o caso

subsidy_band_hint: faixa provável sem prometer resultado

F7 — Jornada e intenção operacional

current_intent: entender programa, seguir análise, enviar docs, visita

docs_channel_choice: WhatsApp, site, visita presencial

visit_interest: sim/não/talvez

stalled_reason: travou por tempo, medo, docs, curiosidade etc.

F8 — Documentos e evidências

doc_identity_status: faltando, parcial, recebido, validado

doc_income_status: faltando, parcial, recebido, validado

doc_residence_status: faltando, parcial, recebido, validado

doc_ctps_status: faltando, parcial, recebido, validado

F9 — Sinais cognitivos controlados

customer_confusion_level: baixo, médio, alto

urgency_signal: baixo, médio, alto

trust_signal: baixo, médio, alto

offtrack_type: curiosidade, objeção, desabafo, pergunta lateral

4. Estado canônico do lead

O estado canônico é uma visão operacional resumida e governada do caso. Ele não replica todos os facts; ele aponta situação atual, objetivo corrente, riscos, pendências, elegibilidade provável e próxima ação permitida.

Campo	Função
lead_id	Identificador estável do lead
current_phase	macro-fase atual: discovery, qualificação, docs,

	correspondente, visita
current_objective	o que precisa ser resolvido agora
process_mode	solo, conjunto, composicao_familiar
progress_score	percentual operacional de completude
risk_level	baixo, médio, alto, crítico
must_ask_now	lista curta de perguntas obrigatórias antes de avançar
blocked_by	motivos formais de bloqueio
recommended_next_actions	ações permitidas em ordem
open_contradictions	facts conflitantes aguardando confirmação
last_policy_decision	último evento de governança relevante
handoff_readiness	none, parcial, pronto para docs, pronto para correspondente, pronto para visita

4.1 Fases macro recomendadas

- discovery: entendimento inicial, explicação do programa, enquadramento do lead.
- qualification: estado civil, composição, trabalho, renda, dependente, restrição, elegibilidade documental.
- docs_prep: escolha de canal e preparação para envio de documentos.
- docs_collection: recebimento e consolidação documental.
- broker_handoff: preparação do pacote ao correspondente.
- visit_conversion: convite, agendamento e acompanhamento de visita quando aplicável.

5. Política de memória

Memória não é dump infinito de conversa. O novo modelo usa memória híbrida: curta para o fluxo imediato, factual para governança e executiva para reentrada. O objetivo é preservar contexto útil sem estourar custo, latência ou ruído.

Camada	Conteúdo	Objetivo
Curto prazo	Últimos turnos e eventos imediatos	Manter coesão conversacional local.

Factual estruturada	Facts persistidos e state atual	Garantir decisão consistente e auditável.
Snapshot executivo	Resumo do caso em linguagem operacional	Reentrada rápida, handoff e retomada segura.
Histórico frio	Turns e artifacts completos	Auditoria, analytics e reprocessamento.

5.1 Regras de snapshot

- Criar snapshot a cada marco relevante: mudança de modo de processo, fechamento de qualificação, entrada em docs, pronto para handoff.
- Snapshot deve conter: resumo do perfil, fatos confirmados, pendências, riscos, bloqueios, próxima ação obrigatória e última decisão de policy.
- Snapshot nunca deve prometer aprovação; ele resume probabilidade operacional, não resultado bancário.
- Se houver contradição crítica, snapshot registra a pendência e não tenta “resolver sozinho”.

6. Política de atualização de facts e estado

Toda atualização deve respeitar status e confiança. O modelo não pode sobrescrever um fato confirmado por palpite fraco. Quando há ambiguidade, o sistema registra conflito e puxa confirmação.

Status do fact	Definição
captured	extraído do turno, ainda não confirmado
confirmed	confirmado pelo cliente ou por evidência forte
inferred	inferido por regra/combinação, com marca explícita
contradicted	entrou em choque com outro fato
obsolete	foi substituído por nova confirmação válida

6.1 Regras canônicas de escrita

- Informação sensível de alta relevância (estado civil, modo de processo, renda, IR, RNM, restrição) só vira confirmed com evidência suficiente.
- Se o cliente responder de forma ambígua, gravar captured/inferred e marcar needs_confirmation.
- Mudança de process_mode deve disparar revisão automática de must_ask_now.
- Mudança em renda/regime deve reexecutar policies ligadas a composição, dependente e viabilidade.

- Facts derivados não apagam fatos brutos; ambos coexistem com proveniência.

7. Modelo de dados recomendado no Supabase

Tabela	Papel
enova_lead_state_v2	Estado atual resumido e governado do lead
enova_facts	Fatos atômicos com status, confiança e fonte
enova_turns	Cada turno completo do atendimento
enova_policy_events	Ações de regra e decisões de governança
enova_memory_snapshots	Resumos executivos do caso
enova_artifacts	Áudio, imagem, sticker interpretado, docs e derivados
enova_prompt_registry	Versões do contrato/prompt ativas por escopo
enova_eval_runs	Validação offline/online e regressões

7.1 Chaves e índices mínimos

- Todas as tabelas operacionais devem indexar lead_id e updated_at.
- Facts devem ter unicidade lógica por lead_id + fact_key + versão ativa, permitindo histórico.
- Policy events devem indexar rule_code, outcome e severity para analytics e debugging.
- Turns devem indexar channel_type, input_type, model_name e latency buckets.

8. Contrato obrigatório de saída do LLM

A Enova pode responder em linguagem humana, mas o backend não deve receber apenas texto solto. Cada turno deve produzir uma estrutura válida para persistência, policy e analytics. O texto bonito fica na superfície; por trás existe contrato.

Campo de saída	Finalidade
reply_text	Resposta final visível ao cliente
facts_extracted	facts capturados neste turno
facts_updated	facts confirmados/alterados
current_objective	objetivo que o agente está perseguindo

next_intended_action	o que pretende fazer em seguida
needs_confirmation	lista de pontos ambíguos
must_ask_now	perguntas obrigatórias abertas
policy_flags	códigos de políticas acionadas
risk_flags	sinais de risco operacional ou de compliance
memory_delta	o que entra no snapshot/estado após o turno

8.1 Regra de ouro

O LLM pode sugerir. Quem consolida avanço operacional é a combinação entre state + facts + policy. Se houver conflito, prevalece a governança, não a eloquência da resposta.

9. Política de consistência e antideriva

- Nenhum fact crítico pode existir sem provenance: turno, fonte ou regra derivadora.
- Toda contradição relevante deve ser explicitada ao operador/sistema, nunca mascarada.
- Toda policy que bloquear ou obrigar ação deve gerar evento persistido.
- Todo rollback de prompt ou policy deve manter compatibilidade com schema ativo.
- Expansões futuras entram primeiro como facts e policies novas; só depois, se preciso, ganham novas fases macro.

10. Ordem dos próximos passos

Ordem	Próximo passo
1	Congelar taxonomia v1 e schema base
2	Implementar persistência mínima de state/facts/turns/policy events
3	Acoplar contrato de saída do LLM
4	Ativar snapshots e datasets de validação
5	Abrir rollout governado pelo PDF 7

ENOVA LLM-FIRST

PDF 7 — Ordem Executiva e Contrato Rígido de Execução

Versão revisada com stack operacional real, cronograma solo-calibrado e janela acelerada de implantação.

Status do documento	Escopo desta revisão	Premissa operacional
ATUALIZADO	Corrigir lacunas do PDF 7 original	Operação solo com IA; objetivo é chegar à produção o mais rápido possível sem trocar velocidade por caos

Nota executiva. Este documento fecha duas pendências críticas levantadas na revisão: (1) definição da stack real de execução do LLM e da convivência com o Worker atual durante a migração; (2) recalibração do cronograma para a sua operação real, sem squad separado, sem paralelismo artificial e sem pressupor produção já em andamento.

1. Objetivo desta atualização

Atualizar a ordem executiva para refletir infraestrutura real, ritmo real e decisão real de implantação. O ponto central é substituir dois motores legados — o mecânico e o cognitivo acoplado — por uma arquitetura LLM-first com governança, preservando velocidade de evolução sem abrir mão das amarras de negócio.

Bloco	Decisão fechada	Por quê	Obrigatoriedade
Stack	Worker continua vivo como orquestrador e policy gateway; LLM roda pela API da OpenAI	Mantém controle, estado e integração sem deixar o modelo solto	Mandatório
Estado	Supabase vira fonte única da verdade do lead	Centraliza facts, memória, policy events, turns e avaliação	Mandatório
Migração	Cutover em ondas curtas, sem reescrever tudo antes de testar	Troca brusca aumenta risco e atrasa	Mandatório
Prazo	Janela acelerada de 10 a 14 dias corridos	Compatível com operação solo intensiva; menor que isso vira aposta cega	Mandatório

2. Stack operacional real do novo modelo

A arquitetura executável recomendada para a Enova é enxuta, mas não simplista. O Worker atual não deve ser descartado; ele deve ser rebaixado de cérebro para controlador. O LLM assume a condução conversacional, porém sempre cercado por estado estruturado, policy engine e persistência forte.

2.1 Arquitetura-alvo mínima

[Canal de entrada / WhatsApp / mídia]

↓

[Cloudflare Worker — orquestrador leve]

↓

[OpenAI API — agente Enova LLM-first]

↓

[Policy Engine + reconciliador de estado]

↓

[Supabase — estado, memória, telemetria e auditoria]

2.2 Papel fechado de cada componente

Componente	Responsabilidade primária	Não deve fazer	Observação
Worker	Receber evento, montar contexto mínimo, chamar LLM, aplicar policies, persistir e responder ao canal	Voltar a carregar o trilho mecânico inteiro como cérebro principal	É o ponto de integração estável durante a migração
OpenAI API	Interpretar, responder natural, extrair facts, sugerir próximo passo, lidar com áudio/imagem/sticker	Persistir direto no banco ou decidir negócio sem validação	O modelo precisa responder em formato estruturado + texto ao cliente
Policy engine	Checar regras obrigatórias, bloquear avanço indevido, exigir pergunta obrigatória ou confirmação	Virar um novo funil gigante rígido	Deve ser declarativo e curto
Supabase	Guardar lead_state_v2, facts, turns, memory snapshots, policy events, eval runs	Ser espelho cego de respostas textuais sem estrutura	É a memória operacional do sistema

2.3 Convivência com o Worker atual durante a migração

- O Worker atual continua sendo a porta única de entrada e saída até o cutover final.
- O motor mecânico legado não deve ser deletado logo de início; ele fica congelado como fallback e referência de comparação por curto período.
- O motor cognitivo atual não deve ser expandido. Ele entra em modo depreciação assim que o novo agente LLM-first passar a operar em shadow com policy e persistência válidas.
- Toda chamada real continua passando pelo mesmo domínio/fluxo do Worker para não explodir integrações do CRM, webhooks e canal de atendimento.

2.4 Decisão sobre serviço separado

Para esta fase, não é obrigatório criar um microserviço separado para o LLM. O menor caminho seguro é manter o orchestration loop dentro do Worker e consumir a OpenAI API diretamente dali. Serviço separado só entra depois, se volume, latência, observabilidade ou governança exigirem.

3. Regras de negócio: como ficam obrigatórias sem voltar ao inferno mecânico

A trava não fica mais escondida em stage hardcoded. Ela passa para três camadas: facts estruturados, policy rules declarativas e reconciliador pós-resposta. Conversa livre; decisão governada.

Regra	Expressão operacional	Ação obrigatória
Solo renda baixa	se processo=solo e renda_total_confirmada < limite_operacional	sugerir composição antes de inviabilizar ou encerrar
Autônomo	se regime_trabalho=autonomo	perguntar formalização/IR obrigatoriamente
Casado civil	se estado_civil=casado_civil	forçar processo_conjunto
Estrangeiro	se nacionalidade!=brasileiro	validar RNM antes de avançar
Dependente	se processo_conjunto=true	não abrir pergunta de dependente; se solo e abaixo da faixa aplicável, abrir quando couber

A regra precisa ser expansiva: novas políticas entram como novos predicates + actions, sem redesenhar o cérebro. O segredo é não prender a fala; prender a governança.

4. Cronograma recalibrado para operação solo e implantação acelerada

Você não está em produção ainda e já carrega oito meses de projeto. Então o plano não pode pressupor uma janela longa, mas também não pode vender milagre. A janela realista, agressiva e ainda defensável é de 10 a 14 dias corridos, com dedicação pesada, noites se necessário e escopo disciplinado. Menos que isso só funciona se você aceitar reduzir o escopo do primeiro go-live.

4.1 Janela executiva recomendada

Onda	Dias	Objetivo	Entrega concreta	Gate
O1	D1–D2	Fechar contrato técnico do agente	System prompt canônico, output JSON obrigatório, tabela inicial de policies, desenho final do estado v2	Sem esse pacote não codar
O2	D2–D4	Subir base nova sem quebrar entrada	Tabelas Supabase, camada de persistência,	Persistência + chamada LLM funcionando

Onda	Dias	Objetivo	Entrega concreta	Gate
			wrapper OpenAI no Worker, logs/telemetria nova	
O3	D4–D6	Shadow operativo	LLM respondendo em paralelo, facts e policies gravados, comparação com comportamento esperado	Shadow estável em casos-base
O4	D6–D9	LLM-first controlado	Policy engine ativo, novo agente assumindo o topo e coleta principal, legado como fallback restrito	Casos críticos passam
O5	D9–D12	Go-live mínimo	Fluxo prioritário completo rodando pelo novo modelo para entrada real controlada	Checklist mínimo de produção
Reserva	D12–D14	Ajustes finos	Correção de bordas, desempenho, prompts e memória	Sem bloqueio severo aberto

4.2 Leitura honesta do prazo

- Dá para acelerar muito porque você ainda não tem produção rodando; isso elimina parte do peso de rollback em usuários reais.
- Mesmo assim, o menor prazo aceitável depende de foco total no caminho crítico: topo, qualificação, regras obrigatórias, estado, memória e handoff documental mínimo.

- Áudio, sticker, multimodal avançado, otimização fina e expansão de CRM entram preferencialmente logo após o go-live mínimo, a menos que algum item seja indispensável ao primeiro atendimento real.
- Virar noite pode encurtar calendário, mas não substitui gate técnico. Pressa sem gate só move o erro para mais perto da produção.

5. Microetapas mandatórias por onda

O1 — Fechamento do contrato técnico (D1–D2)

- Congelar o escopo do primeiro go-live: quais fases entram e quais ficam explicitamente fora.
- Fechar o contrato do agent: identidade, não negociáveis, output estruturado, campos obrigatórios, tratamento de dúvida e contradição.
- Fechar a tabela de policy rules v1 com prioridade alta: renda baixa, autônomo/IR, casado civil, estrangeiro/RNM, dependente.
- Fechar o schema canônico do estado v2 e dos facts.

O2 — Infra funcional mínima (D2–D4)

- Criar ou migrar tabelas do Supabase para state v2, facts, turns, policy events, memory snapshots e eval runs.
- Adicionar ao Worker um wrapper limpo para chamada OpenAI, com timeout, retries curtos e logging de falha.
- Criar o reconciliador: receber saída do LLM, validar schema, aplicar policies, persistir delta e produzir resposta final.
- Garantir que o canal externo continue falando com o mesmo Worker e o mesmo CRM, sem exigir refatoração paralela do ecossistema.

O3 — Shadow operativo (D4–D6)

- Rodar casos-base completos em shadow, sem impacto externo, registrando o que o LLM respondeu, o que extraiu e quais policies dispararam.
- Conferir se facts críticos estão sendo extraídos com coerência e se o modelo não pula obrigação.
- Ajustar prompt, output e policies até o comportamento-base estabilizar.

O4 — Assunção controlada (D6–D9)

- Habilitar o novo agente no topo do funil e na qualificação prioritária.
- Manter fallback legado somente onde ainda não há confiança suficiente.
- Rodar bateria realista de borda: autônomo sem IR, composição, estrangeiro sem RNM, casado civil, cliente fora de trilho.

O5 — Go-live mínimo (D9–D12)

- Abrir produção controlada para o fluxo priorizado.
- Monitorar telemetria, policy violations, latência e respostas inconsistentes.
- Corrigir apenas o que for comprovadamente ativo; não abrir frentes paralelas por ansiedade.

6. Ordem rígida de execução e gates não negociáveis

- Gate G1 — Contrato técnico fechado: sem prompt canônico, schema de saída e policies v1, nenhuma implementação começa.
- Gate G2 — Persistência e OpenAI wrapper válidos: sem gravação de state v2 e facts, shadow não começa.
- Gate G3 — Shadow estável: sem casos-base coerentes e sem policy enforcement mínimo, o LLM não assume o topo.
- Gate G4 — Bateria crítica aprovada: sem passar nos casos de maior risco de negócio, não há go-live.
- Gate G5 — Telemetria e fallback operacional: sem evidência mínima, não se corta o legado de vez.

Qualquer compressão de prazo deve respeitar a ordem dos gates. Pode encurtar a janela entre um gate e outro, mas não pode inverter ou suprimir gate por exaustão.

7. Critério de produção mínima

Bloco	Condição mínima	Status exigido
Canal	Entrada real passa pelo Worker novo sem quebrar integração com CRM	Obrigatório
Governança	Policies críticas disparam e bloqueiam decisões erradas	Obrigatório
Memória	Facts e state v2 persistem de forma consistente por turno	Obrigatório
Telemetria	Logs de turno, policy events e snapshots acessíveis para diagnóstico	Obrigatório
Latência	Tempo de resposta aceitável para atendimento real; se alto, resposta intermediária e otimização pós-go-live	Obrigatório
Borda	Casos de risco principal testados	Obrigatório

8. Fechamento executivo

A correção do PDF 7 fecha o que faltava: stack real e cronograma real. A decisão estratégica continua sendo migrar para um modelo LLM-first com governança. A decisão operacional, agora explicitada, é fazer isso com o Worker ainda vivo como orquestrador, OpenAI como cérebro conversacional, policy engine mínimo como trava de negócio e Supabase como memória operacional. O calendário não é mais de squad idealizado; é uma corrida agressiva de 10 a 14 dias, em operação solo, para chegar a um go-live mínimo realista e defendível.

ENOVA LLM-FIRST v1

PDF 9 — Contrato do Structured Output + Prompt Canônico de Produção

Documento normativo para o formato obrigatório de resposta do agente, prompt-base de produção, regras de execução e compatibilidade GPT-4 → GPT-5.

Resumo executivo: este contrato transforma o modelo em motor operacional. Ele define exatamente como o LLM deve responder, quais campos são obrigatórios, quando deve pedir confirmação, o que nunca pode afirmar e como o Worker deve ler o resultado.

1. Finalidade do contrato

Este documento fecha a camada que faltava entre o prompt do agent e o Policy Engine. Seu objetivo é padronizar o comportamento do modelo em produção para que a ENOVA converse com naturalidade, mas devolva ao sistema uma resposta legível, auditável e utilizável. O LLM deixa de ser apenas um chat e passa a ser um componente com contrato formal de entrada e saída.

2. Escopo e não negociáveis

- O agente deve responder ao cliente em linguagem natural, humana, objetiva e orientada à próxima ação.
- O agente deve sempre devolver saída estruturada no formato definido neste documento.
- O agente nunca pode prometer aprovação, subsídio, imóvel específico, parcela final ou condição bancária final.
- Em caso de ambiguidade ou contradição factual, o agente não pode avançar silenciosamente; deve sinalizar confirmação obrigatória.
- O contrato deve funcionar primeiro com GPT-4 e permanecer compatível com GPT-5 sem redesign estrutural.

3. Papel deste contrato dentro da arquitetura

```
cliente → Worker → prompt + estado + memória → OpenAI
       → structured output obrigatório → Policy Engine
       → persistência no Supabase → resposta final ao cliente
```

4. Prompt canônico de produção

4.1 System prompt-base

```
Você é a ENOVA, especialista em financiamento imobiliário no programa Minha Casa Minha Vida.

Seu papel é conduzir o cliente com naturalidade, clareza e direção, coletando informações necessárias
para análise de financiamento sem quebrar as regras de negócio do sistema.

Você não é um chat livre. Você é um agente operacional.
```

```
Regras gerais:
- Nunca invente informação.
- Nunca prometa aprovação, parcela final, taxa final, valor final de subsídio ou imóvel garantido.
- Nunca ignore informação crítica já confirmada.
- Nunca pule coleta obrigatória quando faltar dado sensível para decisão.
- Sempre conduza a conversa para a próxima ação.
- Sempre que houver ambiguidade, contradição ou baixa confiança, sinalize needs_confirmation = true.
- Se o cliente sair do assunto, responda com naturalidade e traga de volta ao objetivo atual.
- Use linguagem humana, simples, acolhedora e objetiva.
- Evite jargão técnico desnecessário.
- Não fale como robô.
- Não deixe a conversa aberta ou sem direção.
- Use o estado atual do lead como fonte prioritária de verdade.
- Use a memória resumida apenas como apoio contextual.
- Responda sempre no formato estruturado obrigatório definido pelo sistema.
```

4.2 Bloco de regras de negócio mínimas

```
Regras críticas da ENOVA:
1. Casado no civil → processo obrigatoriamente em conjunto.
2. União estável → pode seguir solo ou conjunto, dependendo do contexto e da estratégia.
3. Solteiro → pode seguir solo ou composição; não presumir parceiro.
4. Autônomo → perguntar obrigatoriamente sobre Imposto de Renda quando a informação ainda não
existir.
5. Autônomo sem IR → não encerrar de forma seca; orientar e/ou sugerir composição quando fizer
sentido.
6. Renda solo baixa → sugerir composição antes de inviabilizar.
7. Estrangeiro → exige RNM válido para avançar com segurança.
8. Dependente → só perguntar quando realmente fizer sentido no caso.
9. CTPS 36 meses → informar valor estratégico, mas não travar o fluxo.
10. Sempre reforçar que aprovação depende da análise do perfil e avaliação do banco.
```

4.3 Bloco de tom e superfície de resposta

```
Tom da ENOVA:
- humana
- profissional
- acolhedora
- consultiva
- firme sem ser fria
- objetiva sem parecer brusca

Superfície:
- respostas curtas a médias
- sem parede de texto
- com direção clara
- sempre fechando em próxima ação
```

5. Structured output obrigatório

Toda resposta do modelo deve sair em objeto estruturado. O texto ao cliente é apenas um dos campos. Os demais campos são parte do contrato operacional.

5.1 Schema canônico v1

```
{
  "reply_text": "string",
  "intent": "coleta_dado | resposta_duvida | confirmacao | direcionamento | orientacao",
  "facts_extracted": {
    "estado_civil": "solteiro | uniao_estavel | casado_civil | null",
    "processo": "solo | conjunto | composicao_familiar | null",
    "renda_principal": 0,
    "regime_trabalho": "clt | autonomo | servidor | aposentado | null",
    "autonomo_tem_ir": true,
    "nacionalidade": "brasileiro | estrangeiro | null",
```

```
    "rnm_status": "valido | invalido | ausente | null",
    "dependente_qtd": 0,
    "ctps_36": true
  },
  "facts_updated": [],
  "current_objective": "string",
  "next_step": "string",
  "needs_confirmation": false,
  "confidence": 0.0,
  "policy_flags": [],
  "notes_for_system": "string"
}
```

5.2 Significado obrigatório dos campos

Campo	Obrigatório	Descrição	Regra de uso
reply_text	sim	Mensagem final ao cliente	Sempre natural e direcionada
intent	sim	Natureza da resposta	Escolher valor do enum fechado
facts_extracted	sim	Fatos inferidos do turno	Somente preencher o que tiver base
facts_updated	sim	Lista textual curta dos fatos alterados	Usado para auditoria
current_objective	sim	Objetivo atual da coleta	Deve refletir o foco do turno
next_step	sim	Próxima ação pretendida	Pode ser corrigida pelo Policy Engine
needs_confirmation	sim	Se há dúvida ou ambiguidade	Marcar true em incerteza real
confidence	sim	Nível de confiança do turno	Faixa 0.0–1.0
policy_flags	sim	Sinais de risco ou regra tocada	Mesmo vazio, deve existir
notes_for_system	não	Observação interna curta	Nunca vai ao cliente

5.3 Exemplo de saída válida

```
{
  "reply_text": "Perfeito. Para eu te orientar certo, me confirma uma coisa: você trabalha
registrado, é autônomo ou tem outra forma de renda?",
  "intent": "coleta_dado",
  "facts_extracted": {
    "estado_civil": "solteiro",
    "processo": "solo",
    "renda_principal": 2400,
    "regime_trabalho": null,
    "autonomo_tem_ir": null,
    "nacionalidade": "brasileiro",
    "rnm_status": null,
    "dependente_qtd": null,
    "ctps_36": null
  },
  "facts_updated": ["estado_civil=solteiro", "processo=solo", "renda_principal=2400",
"nacionalidade=brasileiro"],
  "current_objective": "identificar regime de trabalho principal",
  "next_step": "perguntar_regime_trabalho",
  "needs_confirmation": false,
  "confidence": 0.88,
  "policy_flags": [],
  "notes_for_system": "renda solo baixa; avaliar composição após identificar regime"
}
```

6. Regras de preenchimento do structured output

- Nunca preencher fato não dito ou não sustentado pelo turno e pelo estado.
- Se o cliente responder parcialmente, preencher apenas a parte confirmada e manter o restante como null.
- Se houver choque entre o turno atual e um fato confirmado do estado, não sobrescrever silenciosamente; marcar needs_confirmation.
- Confidence não é estética. Deve refletir segurança real de interpretação.
- Notes_for_system deve ser curta, objetiva e operacional.

7. Política de confirmação e ambiguidade

A produção real da ENOVA vai lidar com resposta truncada, ambígua, incompleta e contraditória. Este contrato fecha o comportamento nessas bordas.

```
Marcar needs_confirmation = true quando:
- houver duas leituras plausíveis da mesma resposta
- o cliente disser algo conflitante com estado já confirmado
- o modelo não tiver confiança suficiente para atualizar um fato crítico
- a decisão seguinte depender de um detalhe ausente
```

8. Política de superfície: o que vai ao cliente

- Reply_text deve ser sempre entendível por leigo.
- A mensagem deve ser orientada à ação e fechar o turno pedindo ou confirmando algo.
- Perguntas devem priorizar a informação que desbloqueia o caso, não curiosidade secundária.
- Quando responder dúvida fora do trilho, deve responder e puxar de volta ao objetivo atual.
- Nunca despejar todas as regras do programa de uma vez.

9. Configuração-base do modelo

Parâmetro	GPT-4 (inicial)	GPT-5 (evolução)
Papel na arquitetura	modelo de produção inicial	upgrade para bordas críticas ou substituição total
Temperatura	0.2–0.4	0.2–0.3
Uso recomendado	rodar o sistema e estabilizar comportamento	reduzir drift e melhorar ambiguidade
Necessidade de guardrails	alta	média
Mudança estrutural	nenhuma	nenhuma

10. Compatibilidade GPT-4 → GPT-5

O contrato é model-first, não model-locked. Isso significa que o schema, o prompt, o Policy Engine e o estado estruturado não precisam ser refeitos quando houver migração de GPT-4 para GPT-5. A troca correta deve ocorrer como mudança controlada de modelo, com bateria de casos e comparação de outputs.

- Primeira produção entra com GPT-4 forte para validar comportamento real.
- Após estabilização, o roteador pode subir casos ambíguos para GPT-5.
- O upgrade futuro não pode quebrar schema, enums, policy_flags nem fluxo de persistência.

11. Checklist obrigatório de implantação

- Prompt armazenado com versionamento explícito.
- Schema de saída validado antes de qualquer persistência.
- Fallback para resposta segura quando o LLM retornar fora do contrato.
- Logs do raw output e do normalized output.
- Teste de casos reais com autônomo, renda baixa, estrangeiro e estado civil conjunto.

12. Cláusula de fallback seguro

```
Se o modelo retornar fora do schema:
1. não persistir fatos
2. registrar erro estruturado
3. devolver resposta segura de contingência
4. manter objetivo atual
5. tentar novamente apenas se a política de retry permitir
```

13. Fechamento executivo

Este contrato é o que torna o novo motor utilizável em produção. Sem ele, o LLM fala; com ele, o LLM opera. Ele fecha o ponto de contato entre conversa natural, controle de regras, persistência em base e auditoria de decisão. A partir daqui, o sistema já pode avançar com pipeline de produção real, roteamento por modelo e validação por bateria de casos.

ENOVA LLM-FIRST v1

PDF 10 — Contrato do Roteador de Modelos + Política de Escalonamento GPT-4 → GPT-5

Documento normativo para seleção de modelo, escalonamento por criticidade, custo, fallback e governança do uso de GPT-4 e GPT-5.

Resumo executivo: a ENOVA não deve tratar todos os turnos como iguais. Este contrato define quando usar GPT-4 como padrão, quando subir para GPT-5, quando segurar custo, quando bloquear resposta e como registrar cada decisão de roteamento.

1. Finalidade do contrato

Este documento fecha a política de escolha de modelo na arquitetura LLM-first. Ele define o roteador de modelos como componente obrigatório entre o Worker e a chamada ao provedor, com o objetivo de equilibrar precisão, custo, latência e risco operacional. O roteador não decide regras de negócio; ele decide qual nível de modelo deve ser usado para cada turno.

2. Princípios canônicos

- GPT-4 forte entra como modelo padrão inicial de produção.
- GPT-5 não entra por vaidade; entra por necessidade operacional clara.
- Turnos simples não devem consumir modelo caro sem justificativa.
- Casos ambíguos, contraditórios ou de alto risco podem subir para GPT-5.
- Toda decisão de roteamento deve ser registrada com motivo explícito.
- Fallback de modelo nunca pode quebrar schema, policy engine ou persistência.

3. Papel do roteador na arquitetura

```
cliente → Worker
       → carregar estado + memória + objetivo atual
       → classificar criticidade do turno
       → roteador de modelos
           → GPT-4 (default)
           → GPT-5 (escalado)
       → structured output
       → Policy Engine
       → persistência + resposta final
```

4. Política-base de uso dos modelos

Camada	Modelo	Uso canônico	Observação
Default inicial	GPT-4	maior parte dos turnos de coleta, continuidade e resposta controlada	modelo padrão de produção inicial
Escalonamento crítico	GPT-5	ambiguidades, contradições, composição complexa,	uso seletivo

		múltiplos sinais	
Contingência	GPT-4	quando GPT-5 falhar ou estiver indisponível	deve manter schema
Shadow/eval	GPT-5 paralelo	comparação controlada de qualidade antes de ampliar uso	sem impactar cliente se não aprovado

5. Classificação de criticidade do turno

5.1 Classes oficiais

- C0 — turno simples: resposta curta, coleta direta, baixo risco interpretativo.
- C1 — turno moderado: cliente respondeu parcialmente, mas sem conflito relevante.
- C2 — turno sensível: há possibilidade real de interpretação errada com impacto em regra.
- C3 — turno crítico: contradição, múltiplas leituras plausíveis, composição complexa ou bloqueio relevante.

5.2 Heurísticas mínimas de classificação

```
Subir o turno para C2 ou C3 quando houver um ou mais dos sinais abaixo:
- baixa confidence do último turno
- needs_confirmation = true
- conflito entre fato extraído e estado confirmado
- múltiplas rendas ou múltiplos regimes no mesmo texto
- composição familiar complexa
- estrangeiro + documentação pendente
- pergunta fora do trilho com impacto em decisão
- histórico recente de falha de entendimento
```

6. Política de roteamento

6.1 Regra executiva

```
C0 → GPT-4
C1 → GPT-4
C2 → GPT-4 ou GPT-5, conforme score de risco
C3 → GPT-5
```

6.2 Score mínimo de risco

```
score = 0

if last_turn.needs_confirmation: score += 3
if last_turn.confidence < 0.65: score += 2
if state.has_recent_contradiction: score += 3
if state.multiple_income_signals: score += 2
if state.multiple_regime_signals: score += 2
if state.foreign_document_pending: score += 3
if state.complex_family_composition: score += 3
if user_message_is_offtrack_but_sensitive: score += 2

if score <= 2:
  route = "gpt4"
elif score <= 5:
  route = "gpt4_or_gpt5_by_budget"
```

```
else:
  route = "gpt5"
```

7. Política de orçamento e custo

- O sistema deve ter budget diário e budget por lead para não escalar sem controle.
- GPT-5 não deve virar default por conveniência emocional; deve ter gatilho técnico.
- Em períodos de alto volume, o limiar de escalonamento pode subir levemente, sem comprometer casos críticos.
- Quando o budget crítico for atingido, o sistema deve priorizar GPT-5 apenas para C3.

8. Política de latência

- Turnos C0 e C1 devem priorizar fluidez e tempo de resposta menor.
- Turnos C2 e C3 podem aceitar latência maior em troca de menor erro.
- O roteador deve registrar se houve demora acima do esperado para recalibração posterior.
- Latência nunca pode justificar quebra de schema ou resposta solta.

9. Código-base do roteador

9.1 Função de classificação

```
export function classifyTurnRisk({ state, lastTurn, userMessage }) {
  let score = 0;
  const reasons = [];

  if (lastTurn?.needs_confirmation) {
    score += 3;
    reasons.push("needs_confirmation");
  }

  if ((lastTurn?.confidence ?? 1) < 0.65) {
    score += 2;
    reasons.push("low_confidence");
  }

  if (state?.has_recent_contradiction) {
    score += 3;
    reasons.push("recent_contradiction");
  }

  if (state?.multiple_income_signals) {
    score += 2;
    reasons.push("multiple_income_signals");
  }

  if (state?.multiple_regime_signals) {
    score += 2;
    reasons.push("multiple_regime_signals");
  }

  if (state?.foreign_document_pending) {
    score += 3;
    reasons.push("foreign_document_pending");
  }

  if (state?.complex_family_composition) {
    score += 3;
```

```
    reasons.push("complex_family_composition");
  }

  if (isSensitiveOfftrack(userMessage)) {
    score += 2;
    reasons.push("sensitive_offtrack");
  }

  let criticality = "C0";
  if (score >= 6) criticality = "C3";
  else if (score >= 3) criticality = "C2";
  else if (score >= 1) criticality = "C1";

  return { score, criticality, reasons };
}
```

9.2 Função de roteamento

```
export function routeModel({ risk, budget }) {
  if (risk.criticality === "C3") {
    return {
      model: "gpt5",
      reason: "critical_turn"
    };
  }

  if (risk.criticality === "C2") {
    if (budget?.allow_gpt5_for_moderate_risk) {
      return {
        model: "gpt5",
        reason: "moderate_risk_escalated"
      };
    }

    return {
      model: "gpt4",
      reason: "moderate_risk_budget_guard"
    };
  }

  return {
    model: "gpt4",
    reason: "default_path"
  };
}
```

9.3 Orquestração no Worker

```
const risk = classifyTurnRisk({ state, lastTurn, userMessage });
const routing = routeModel({ risk, budget: runtimeBudget });

const llmOutput = await callOpenAI({
  model: routing.model,
  systemPrompt,
  state,
  memory,
  userMessage
});

await logModelRouting({
  leadId: state.lead_id,
  model: routing.model,
  criticality: risk.criticality,
  score: risk.score,
  reasons: risk.reasons,
  routeReason: routing.reason
});
```

10. Política de fallback

```
Se GPT-5 falhar:
1. registrar falha
2. tentar GPT-4 com mesma estrutura de saída
3. marcar route_fallback = true
4. não persistir fatos se o retorno vier fora do contrato

Se GPT-4 falhar:
1. registrar falha
2. tentar retry controlado
3. usar resposta segura se necessário
```

11. Política de shadow e evolução

- Antes de ampliar o uso de GPT-5, rodar shadow comparando GPT-4 vs GPT-5 em casos reais sem impacto no cliente.
- A evolução deve ser orientada por evidência: menos contradição, melhor captura de fatos e menor necessidade de confirmação tardia.
- Se GPT-5 mostrar ganho real e custo aceitável, ampliar progressivamente a classe C2.
- Se não mostrar ganho suficiente, manter GPT-4 como default por mais tempo.

12. Persistência mínima de roteamento

```
Tabela: enova_model_routing_events

Campos mínimos:
- id
- lead_id
- turn_id
- model_selected
- criticality
- risk_score
- risk_reasons
- route_reason
- fallback_used
- latency_ms
- token_input_est
- token_output_est
- created_at
```

13. KPIs oficiais do roteador

- taxa de escalonamento GPT-5 por classe de risco
- custo médio por lead
- latência média por modelo
- taxa de fallback por modelo
- ganho líquido de qualidade em C2/C3
- redução de contradições ou confirmações tardias

14. Janela de implantação

- D1–D2: classificador de risco + logs de roteamento.

- D3–D4: roteador real GPT-4 default / GPT-5 escalado.
- D5–D6: fallback + budget guard.
- D7–D9: shadow GPT-4 vs GPT-5 em casos C2/C3.
- D10–D14: recalibração de limiares e corte operacional.

15. Fechamento executivo

Este contrato impede dois extremos ruins: gastar caro em tudo e confiar barato demais em tudo. A ENOVA passa a ter inteligência econômica de inferência. O sistema usa GPT-4 onde ele basta e sobe para GPT-5 onde o risco justifica. Com isso, custo, precisão e controle deixam de competir cegamente e passam a ser governados.

ENOVA LLM-FIRST v1

PDF 11 — Contrato de Memória Operacional do Lead

Este contrato define como a memória do lead funciona no novo modelo LLM-first.

1. Objetivo

Garantir que o sistema nunca dependa apenas do texto da conversa.

2. Camadas de memória

- Curto prazo: últimos turnos
- Médio prazo: facts estruturados
- Longo prazo: snapshots resumidos

3. Snapshot

Resumo consolidado do lead para reduzir custo de contexto.

4. Regras

- Nunca confiar só no texto
- Sempre usar estado estruturado
- Atualizar somente com confiança

5. Estrutura base

```
{
 estado_civil,
 renda,
 regime_trabalho,
 processo,
 flags
}
```

6. Uso no LLM

Enviar apenas o necessário para reduzir tokens.

ENOVA LLM-FIRST v1

PDF 11 — Contrato de Memória Operacional do Lead + Política de Resumo, Snapshot e Janela de Contexto

Documento normativo para a memória do lead, fonte de verdade operacional, snapshots, contexto enviado ao modelo e regras de atualização segura.

Resumo executivo: este contrato impede que a ENOVA dependa apenas do histórico bruto da conversa. Ele define qual memória vale, como fatos são promovidos, quando resumir, o que enviar ao modelo e como preservar custo, coerência e governança.

1. Finalidade do contrato

Este documento fecha a política de memória operacional da arquitetura LLM-first. Seu objetivo é garantir que cada lead tenha uma fonte única de verdade, separando claramente histórico bruto, fatos confirmados, sinais ambíguos, snapshot resumido e contexto de inferência. Sem esse contrato, o sistema tende a inflar custo, repetir perguntas, aceitar contradições silenciosas e perder consistência ao longo do atendimento.

2. Princípios canônicos

- Histórico bruto não é fonte de verdade; é apenas evidência.
- Fato confirmado vale mais que lembrança solta do modelo.
- Informação ambígua não pode virar fato consolidado sem confirmação.
- A memória deve reduzir custo de contexto, não aumentar dependência de tokens.
- O modelo deve receber apenas o contexto necessário para decidir o turno atual.
- Toda promoção de memória deve ser auditável e reversível.

3. Camadas oficiais de memória

Camada	Função	Persistência	Uso principal
M0 — histórico bruto	registro dos turnos originais	enova_turns	auditoria e reprocessamento
M1 — fatos estruturados	estado operacional atual	enova_lead_state_v2 + enova_facts	fonte de verdade do lead
M2 — memória ambígua	sinais ainda não confirmados	enova_facts com status=ambiguous	espera de confirmação
M3 — snapshot resumido	resumo de contexto para o LLM	enova_memory_snapshots	redução de tokens e continuidade

4. Fonte única da verdade

A fonte única da verdade do lead é a combinação de estado estruturado atual e fatos confirmados. O snapshot é derivado. O histórico bruto é suporte. O modelo nunca deve tratar o histórico como superior ao estado.

4.1 Ordem de precedência

```
1. enova_lead_state_v2 (estado canônico atual)
2. enova_facts com status = confirmed
3. enova_memory_snapshots mais recente
4. enova_turns recentes
5. texto atual do usuário
```

5. Estrutura mínima da memória operacional

```
{
  "lead_id": "string",
  "estado_civil": "solteiro | uniao_estavel | casado_civil | null",
  "processo": "solo | conjunto | composicao_familiar | null",
  "renda_principal": 0,
  "regime_trabalho": "clt | autonomo | servidor | aposentado | null",
  "autonomo_tem_ir": true,
  "nacionalidade": "brasileiro | estrangeiro | null",
  "rnm_status": "valido | invalido | ausente | null",
  "dependente_qtd": 0,
  "ctps_36": true,
  "pending_questions": [],
  "risk_flags": [],
  "has_recent_contradiction": false,
  "current_objective": "string",
  "memory_version": 1
}
```

6. Política de promoção de fatos

Nem tudo o que o cliente diz vira fato confirmado. A promoção de um valor para memória consolidada depende de classe do dado, clareza do turno e ausência de conflito crítico.

6.1 Classes de dados

- Classe A — crítica estrutural: estado civil, processo, nacionalidade, RNM, regime de trabalho.
- Classe B — crítica financeira: renda principal, autônomo com ou sem IR, composição de renda.
- Classe C — complementar: dependentes, CTPS 36 meses, observações úteis.
- Classe D — contextual: preferências, objeções, horário, intenção de visita, tom do cliente.

6.2 Regra de promoção

```
Classe A:
- exige alta confiança
- não sobrescreve fato confirmado sem confirmação

Classe B:
- pode ser promovida com alta ou média-alta confiança
- conflito relevante exige confirmação

Classe C:
- pode entrar com confirmação implícita quando não houver risco estrutural

Classe D:
- pode ser salva como contexto leve sem travar fluxo
```

7. Política de ambiguidade

- Resposta ambígua gera fato com status=ambiguous, não confirmed.
- Fato ambíguo entra em fila de confirmação e pode orientar a próxima pergunta.
- Fato ambíguo não muda roteamento estrutural sozinho.
- Se a ambiguidade tocar regra crítica, needs_confirmation deve virar verdadeiro.

8. Snapshot operacional

O snapshot é um resumo compacto do caso, criado para abastecer o modelo com contexto suficiente sem despejar todo o histórico bruto. Ele deve ser pequeno, legível e fiel ao estado confirmado.

8.1 Conteúdo obrigatório do snapshot

```
{
  "lead_summary": "solteiro, brasileiro, renda 2400, regime ainda não confirmado",
  "confirmed_facts": [
    "estado_civil=solteiro",
    "nacionalidade=brasileiro",
    "renda_principal=2400"
  ],
  "open_questions": [
    "regime_trabalho",
    "autonomo_tem_ir_if_applicable"
  ],
  "risk_flags": [
    "low_solo_income"
  ],
  "current_objective": "identificar regime de trabalho",
  "do_not_forget": [
    "se autonomo, perguntar IR",
    "se renda solo baixa, sugerir composição"
  ]
}
```

8.2 Quando regenerar snapshot

- quando houver mudança de fato Classe A ou B
- quando o histórico recente crescer além da janela padrão
- quando houver resolução de contradição importante
- quando o current_objective mudar de frente relevante

9. Janela de contexto enviada ao LLM

O modelo não deve receber a conversa inteira por padrão. O contrato fecha uma janela de contexto curta, apoiada por snapshot e estado estruturado.

```
Contexto padrão do turno:
1. system prompt canônico
2. objetivo atual
3. estado estruturado atual
4. snapshot mais recente
5. últimos 3 a 6 turnos relevantes
6. mensagem atual do usuário
```

9.1 Regras de economia de contexto

- não enviar blocos antigos irrelevantes

- não repetir fatos já consolidados dentro do histórico se já estão no estado
- não mandar snapshot grande demais
- em caso de custo alto, reduzir histórico antes de reduzir estado

10. Persistência mínima no Supabase

```
Tabela: enova_lead_state_v2
- estado atual canônico do lead

Tabela: enova_facts
- lead_id
- fact_key
- fact_value
- confidence
- status (confirmed | ambiguous | superseded)
- source_turn_id
- created_at
- updated_at

Tabela: enova_memory_snapshots
- id
- lead_id
- summary_json
- generated_from_turn_id
- version
- created_at
```

11. Política de sobrescrita e supersede

- fato confirmado não deve ser apagado; deve ser superseded quando substituído legitimamente
- toda troca crítica deve manter histórico de versão
- se houver conflito com fato confirmado, guardar o novo como pending/ambiguous até confirmação
- não fazer limpeza cega de memória estrutural

12. Código-base do montador de contexto

```
export function buildModelContext({
  systemPrompt,
  state,
  snapshot,
  recentTurns,
  userMessage,
  currentObjective
}) {
  return {
    systemPrompt,
    objective: currentObjective || state.current_objective,
    state,
    snapshot,
    recentTurns: recentTurns.slice(-6),
    userMessage
  };
}
```

12.1 Código-base do gerador de snapshot

```
export function buildLeadSnapshot({ state, facts }) {
  const confirmedFacts = facts
    .filter(f => f.status === "confirmed")
    .map(f => `${f.fact_key}=${String(f.fact_value)}`);

  return {
    lead_summary: summarizeLeadState(state),
    confirmed_facts: confirmedFacts.slice(0, 12),
    open_questions: state.pending_questions || [],
    risk_flags: state.risk_flags || [],
    current_objective: state.current_objective || null,
    do_not_forget: deriveOperationalReminders(state)
  };
}
```

13. KPIs oficiais da memória

- taxa de repetição de pergunta já respondida
- taxa de contradição entre turnos e estado
- tokens médios por turno
- tamanho médio do snapshot
- taxa de fatos ambíguos resolvidos
- percentual de turnos que precisaram reabrir confirmação

14. Janela de implantação

- D1–D2: estruturar estado canônico + facts confirmados/ambíguos.
- D3–D4: persistir snapshots e montar contexto do modelo.
- D5–D6: aplicar política de promoção e sobrescrita segura.
- D7–D9: reduzir custo de contexto e validar repetição de perguntas.
- D10–D14: recalibrar snapshot, janela de turnos e KPIs.

15. Fechamento executivo

Este contrato garante que a memória da ENOVA seja operacional e não fantasiosa. Ele fecha a diferença entre lembrar e saber. O sistema passa a carregar estado verdadeiro, contexto útil e histórico auditável, sem depender de muralha de tokens nem de memória implícita do modelo.

ENOVA LLM-FIRST v1

PDF 12 — Contrato de Telemetria + Observabilidade + Diagnóstico Operacional

Documento normativo para eventos, logs, rastreabilidade, investigação de erro e governança diagnóstica do novo motor.

Resumo executivo: este contrato impede que a ENOVA volte para o escuro. Ele define o que deve ser logado, quais eventos precisam existir, como rastrear uma falha de ponta a ponta, como diferenciar erro de modelo, erro de policy, erro de memória e erro de integração.

1. Finalidade do contrato

Este documento fecha a camada de observabilidade do novo modelo LLM-first. Seu objetivo é garantir diagnóstico rápido, rastreio confiável e leitura operacional do comportamento da ENOVA em ambiente real. Sem esse contrato, qualquer falha vira opinião, impressão ou tentativa e erro.

2. Princípios canônicos

- Toda decisão relevante deve deixar rastro.
- Nenhum erro crítico pode depender apenas de leitura subjetiva do atendimento.
- O diagnóstico sempre começa por telemetria, não por palpite.
- Eventos devem ser legíveis por máquina e auditáveis por humano.
- Logs devem ser suficientes para investigar sem expor além do necessário.
- Toda camada crítica deve ser observável: entrada, roteador, modelo, policy, memória, persistência e saída.

3. Camadas observáveis obrigatórias

Camada	O que observar	Evento mínimo	Objetivo
Entrada	mensagem recebida, canal, lead, request_id	turn.received	prova de entrada
Roteador	modelo escolhido, score, criticidade	model.routed	entender custo e risco
LLM	output bruto, schema ok/erro, latência	llm.completed	validar resposta do modelo
Policy	violações, bloqueios, forced updates	policy.evaluated	explicar correção do fluxo
Memória	facts promovidos, ambiguidades, snapshot	memory.updated	rastrear estado
Persistência	save ok/erro	db.persisted	garantir escrita confiável
Saída	reply final, next_step, flags	turn.responded	fechar trilha ponta a ponta

4. Taxonomia mínima de eventos

```
turn.received
context.built
model.risk_classified
model.routed
llm.requested
llm.completed
llm.schema_invalid
policy.evaluated
policy.blocked
memory.updated
snapshot.generated
db.persisted
db.error
turn.responded
turn.fallback_used
```

5. Contrato mínimo do evento

```
{
  "event_name": "string",
  "lead_id": "string",
  "turn_id": "string",
  "request_id": "string",
  "execution_id": "string",
  "timestamp": "ISO-8601",
  "layer": "entry | routing | llm | policy | memory | db | output",
  "severity": "info | warning | error | critical",
  "payload": {},
  "tags": [],
  "version": 1
}
```

6. Campos mandatórios por evento-chave

6.1 model.routed

```
{
  "model_selected": "gpt4 | gpt5",
  "criticality": "C0 | C1 | C2 | C3",
  "risk_score": 0,
  "risk_reasons": [],
  "route_reason": "default_path | critical_turn | budget_guard | fallback"
}
```

6.2 llm.completed

```
{
  "latency_ms": 0,
  "schema_valid": true,
  "confidence": 0.0,
  "needs_confirmation": false,
  "token_input_est": 0,
  "token_output_est": 0
}
```

6.3 policy.evaluated

```
{
  "ok": true,
  "block_advance": false,
  "violations": [],
  "required_questions": [],
  "forced_updates": {}
}
```

6.4 memory.updated

```
{
  "facts_confirmed": [],
  "facts_ambiguous": [],
  "snapshot_regenerated": false,
  "superseded_facts": []
}
```

7. Tabelas mínimas no Supabase

```
enova_events
- id
- event_name
- lead_id
- turn_id
- request_id
- execution_id
- layer
- severity
- payload_json
- tags
- created_at

enova_incidents
- id
- lead_id
- turn_id
- incident_type
- status
- root_cause
- created_at
- resolved_at
```

8. Código-base do emitter

```
export async function emitEvent(event) {
  const normalized = {
    version: 1,
    timestamp: new Date().toISOString(),
    ...event
  };

  console.log("[ENOVA_EVENT]", JSON.stringify(normalized));

  await supabase.from("enova_events").insert({
    event_name: normalized.event_name,
    lead_id: normalized.lead_id || null,
    turn_id: normalized.turn_id || null,
    request_id: normalized.request_id || null,
```

```
    execution_id: normalized.execution_id || null,
    layer: normalized.layer || "unknown",
    severity: normalized.severity || "info",
    payload_json: normalized.payload || {},
    tags: normalized.tags || []
  });

  return normalized;
}
```

9. Sequência mínima por turno

```
1. emit turn.received
2. emit context.built
3. emit model.risk_classified
4. emit model.routed
5. emit llm.requested
6. emit llm.completed ou llm.schema_invalid
7. emit policy.evaluated
8. emit memory.updated
9. emit db.persisted ou db.error
10. emit turn.responded
```

10. Política oficial de diagnóstico

- Passo 1: localizar lead_id / turn_id / request_id.
- Passo 2: abrir trilha completa de eventos do turno.
- Passo 3: identificar em qual camada o desvio apareceu primeiro.
- Passo 4: confirmar se o problema veio de input, roteamento, LLM, policy, memória ou persistência.
- Passo 5: só depois propor correção cirúrgica.

11. Classificação de falhas

Tipo de falha	Sinal típico	Ação
Falha de schema	llm.schema_invalid	fallback seguro + análise do prompt/output
Falha de policy	policy.blocked indevido ou ausente	revisar regra/condição
Falha de memória	fato repetido, contradito ou esquecido	revisar promoção/snapshot
Falha de roteamento	modelo fraco em turno crítico	ajustar score/threshold
Falha de persistência	db.error	corrigir escrita e retry controlado

12. KPIs oficiais de observabilidade

- tempo médio para identificar camada da falha
- taxa de schema_invalid por modelo
- taxa de fallback por turno
- latência média por camada
- percentual de turnos com trilha completa de eventos
- taxa de incidentes sem causa raiz fechada

13. Política de retenção e limpeza

- manter eventos operacionais suficientes para análise de comportamento e regressão
- separar retenção de eventos detalhados e agregados
- nunca apagar trilha necessária para investigar incidente recente
- resumos agregados podem sobreviver além da janela detalhada

14. Janela de implantação

- D1–D2: subir emitter + tabela enova_events.
- D3–D4: instrumentar entrada, roteador, LLM e policy.
- D5–D6: instrumentar memória e persistência.
- D7–D9: criar visão diagnóstica por lead/turn/request.
- D10–D14: fechar KPIs, incidentes e rotina de leitura.

15. Fechamento executivo

Este contrato fecha a diferença entre sentir que algo deu errado e provar exatamente onde deu errado. Com ele, a ENOVA ganha rastreabilidade ponta a ponta, diagnósticos mais rápidos e correções mais cirúrgicas. É a camada que impede retorno ao escuro.

ENOVA LLM-FIRST v1

PDF 13 — Contrato de Rollout, Shadow Mode, Canary e Cutover para Go-Live

Documento normativo para entrada controlada em operação, comparação entre motores, exposição progressiva, reversão e corte final para produção.

Resumo executivo: este contrato impede que a migração para o novo motor aconteça no escuro. Ele define como comparar o LLM-first com o fluxo anterior, como expor gradualmente atendimento real, como medir risco antes do go-live e como cortar ou reverter sem bagunçar a operação.

1. Finalidade do contrato

Este documento fecha a política de entrada em operação do novo modelo ENOVA LLM-first. Seu objetivo é garantir uma transição controlada entre validação interna, shadow mode, canary e cutover final, sem depender de feeling, sem corte cego e sem empurrar um motor novo para o ambiente real sem leitura objetiva.

2. Princípios canônicos

- Nenhum go-live sério acontece sem critérios de entrada e saída.
- Shadow mode serve para provar comportamento; não é enfeite nem etapa opcional.
- Canary não é abrir geral com medo; é exposição parcial com leitura fina.
- Cutover só acontece quando o novo motor já demonstrou estabilidade suficiente.
- Rollback deve estar definido antes do go-live, não depois.
- Toda fase precisa de métrica, dono e evidência mínima.

3. Fases oficiais do rollout

Fase	Objetivo	Exposição real	Critério de saída	Risco
R0 — preparação	subir infra, contrato, logs e guardrails	0%	pipeline íntegro	baixo
R1 — shadow mode	comparar decisões sem impactar cliente	0% ao cliente	paridade e segurança mínimas	baixo
R2 — canary restrito	deixar subset real responder pelo novo motor	5%–15%	qualidade e estabilidade	médio
R3 — canary ampliado	ampliar exposição progressiva	25%–60%	erro dentro do limite	médio
R4 — cutover	novo motor assume como primário	80%–100%	operação sustentada	alto controlado

4. Shadow mode

No shadow mode, o novo motor roda junto do pipeline atual, mas sua resposta não é enviada ao cliente como decisão oficial. O objetivo é medir aderência às regras, qualidade de extração, estabilidade do schema, comportamento do Policy Engine e custo real.

4.1 O que comparar no shadow

- facts_extracted do novo motor vs leitura esperada do caso
- next_step do novo motor vs objetivo correto
- policy_flags gerados
- taxa de needs_confirmation
- schema_valid, latência e custo por turno
- repetição de pergunta e desvio de regra

4.2 Evidência mínima para sair do shadow

```
Sair do shadow exige, no mínimo:
- schema valid estável
- baixa taxa de contradição factual crítica
- policy engine atuando corretamente
- custo e latência aceitáveis
- bateria de casos críticos sem desvio estrutural relevante
```

5. Canary

Canary é a fase em que parte real dos atendimentos passa a usar o novo motor como resposta efetiva ao cliente. A exposição deve ser controlada por critérios operacionais, não por impulso.

5.1 Política de entrada no canary

- entrar com subset pequeno e observável
- priorizar perfis menos complexos no primeiro recorte, se necessário
- garantir rollback rápido por flag
- monitorar turnos em tempo quase real

5.2 Faixas oficiais de canary

```
Canary 1: 5% dos leads elegíveis
Canary 2: 10% a 15%
Canary 3: 25% a 35%
Canary 4: 50% a 60%
```

6. Critérios de gate por fase

6.1 Gate R0 → R1

- prompt canônico, schema, memory, routing e policy já integrados
- telemetria ponta a ponta funcionando
- persistência mínima validada
- fallback seguro implementado

6.2 Gate R1 → R2

- shadow com trilha suficiente de casos

- sem desvio crítico recorrente em regras sensíveis
- schema_invalid sob controle
- latência/custo aceitáveis para o primeiro canary

6.3 Gate R2/R3 → R4

- canary estável por janela mínima definida
- incidentes críticos zerados ou sob controle claro
- KPI de qualidade sustentado
- plano de rollback validado em teste

7. Modelo de decisão do rollout

```
if fase == "shadow" and gates_ok:
    abrir_canary_1()

if canary.error_rate > limite or policy_break_detected:
    reduzir_exposicao_ou_rollback()

if canary.stable and quality_score >= target:
    ampliar_canary()

if canary_amplo_estavel and rollback_ready:
    executar_cutover()
```

8. Feature flags mínimas

```
Flags mínimas:
- LLM_FIRST_ENABLED
- LLM_FIRST_SHADOW_ENABLED
- LLM_FIRST_CANARY_PERCENT
- LLM_FIRST_MODEL_ROUTER_ENABLED
- LLM_FIRST_POLICY_STRICT_MODE
- LLM_FIRST_FORCE_FALLBACK
```

9. Rollback

Rollback não é sinal de fracasso; é mecanismo obrigatório de segurança. Ele deve ser possível por flag, sem depender de deploy emergencial complexo.

9.1 Disparadores oficiais de rollback

- quebra de regra crítica em atendimento real
- aumento abrupto de schema_invalid ou fallback
- falha persistente de persistência ou memória
- latência inviável para operação real
- desvio grave de classificação em perfil sensível

9.2 Modo de rollback

```
Rollback nível 1:
- reduzir canary para 0%
- manter shadow ligado para diagnóstico
```

```
Rollback nível 2:
- ativar FORCE_FALLBACK
- novo motor deixa de responder ao cliente

Rollback nível 3:
- desligar LLM_FIRST_ENABLED
- operação volta ao caminho anterior até correção
```

10. Cutover

Cutover é o ponto em que o novo motor passa de candidato validado para motor primário. Esse passo exige autorização explícita, leitura de métricas e rollback pronto.

10.1 Checklist obrigatório de cutover

- telemetria íntegra
- feature flags documentadas
- rollback testado
- canary ampliado estável
- alertas mínimos configurados
- janela operacional definida para o corte

11. Tabelas mínimas de rollout no Supabase

```
enova_rollout_events
- id
- phase
- action
- percent_exposed
- lead_scope
- metrics_snapshot
- created_at

enova_rollout_incidents
- id
- phase
- lead_id
- turn_id
- severity
- root_cause
- action_taken
- created_at
```

12. Código-base de decisão de canary

```
export function shouldUseNewEngine({ leadId, canaryPercent, shadowEnabled, forceFallback }) {
  if (forceFallback) return { live: false, shadow: shadowEnabled };
  if (canaryPercent <= 0) return { live: false, shadow: shadowEnabled };

  const bucket = stablePercentBucket(leadId); // 0..99
  const live = bucket < canaryPercent;

  return { live, shadow: shadowEnabled && !live };
}
```

13. KPIs oficiais do rollout

- schema_valid rate
- policy violation rate
- contradição factual crítica
- fallback rate
- latência média por fase
- custo por lead
- repetição indevida de pergunta
- incidentes por 100 leads

14. Janela agressiva de implantação

- D1–D2: subir flags, tabela de rollout e shadow operacional.
- D3–D4: shadow com bateria crítica e leitura diária.
- D5–D6: canary 1 (5%–10%) com monitoramento fechado.
- D7–D8: canary 2/3 se KPIs sustentarem.
- D9–D10: canary ampliado.
- D11–D14: cutover controlado ou manutenção em canary ampliado, conforme evidência.

15. Fechamento executivo

Este contrato fecha a ponte entre arquitetura pronta e operação real. Ele garante que a entrada do novo motor seja tratada como implantação séria, com exposição progressiva, leitura objetiva, rollback limpo e corte controlado. É a camada que transforma pressa em aceleração governada, não em salto no escuro.

ENOVA LLM-FIRST v1

PDF 14 — Contrato de Segurança Operacional + Fail-Safes + Resposta Segura

Este contrato define como o sistema se protege contra falhas do LLM, da lógica e da integração.

1. Objetivo

Garantir que nenhuma falha cause comportamento perigoso ou perda de controle.

2. Tipos de falha

- Falha de LLM (resposta inválida)
- Falha de schema
- Falha de policy
- Falha de memória
- Falha de integração

3. Resposta segura padrão

Sempre retornar uma mensagem neutra e pedir confirmação ao cliente.

Exemplo:
"Quero te orientar certo, então me confirma uma coisa antes da gente seguir..."

4. Regras de segurança

- Nunca avançar com dúvida crítica
- Nunca confiar em output inválido
- Sempre registrar erro
- Sempre manter controle do fluxo

5. Fail-safe

Sistema deve cair para modo seguro em caso de erro.

```
if erro_critico:
  ativar_resposta_segura()
```

bloquear_avanco()

6. Política de retry

- 1 tentativa automática
- depois fallback
- nunca loop infinito

ENOVA LLM-FIRST v1

PDF 15 — Contrato de Testes Reais + Bateria de Casos + Critérios de Aprovação

Este contrato define como validar o sistema antes de entrar em produção.

1. Objetivo

Garantir que o sistema funcione corretamente em cenários reais antes do go-live.

2. Tipos de teste

- Teste unitário (funções)

- Teste de fluxo (etapas do funil)

- Teste de casos reais

- Teste de edge cases

3. Bateria mínima de casos

- Solteiro renda baixa → sugerir composição
- Casado civil → forçar conjunto
- Autônomo sem IR → perguntar IR
- Estrangeiro sem RNM → bloquear avanço
- Cliente confuso → pedir confirmação

4. Critérios de aprovação

- Nenhuma quebra de regra crítica

- Structured output válido

- Policy funcionando

- Sem repetição burra de perguntas

5. Teste real obrigatório

Rodar com conversas reais simuladas antes do go-live.

6. Go/No-Go

Se passar nos critérios → GO

Se falhar em regra crítica → NO-GO

ENOVA LLM-FIRST v1

PDF 16 — Contrato Final de Go-Live + Operação Contínua + Evolução Pós-Produção

Documento executivo para a virada final, rotina operacional diária, critérios de sustentação e trilha oficial de evolução após entrada em produção.

Resumo executivo: este contrato fecha a vida operacional da ENOVA depois que ela entrar no ar. Ele define como autorizar o go-live, como operar no dia seguinte, como reagir a desvio real, como priorizar melhorias e como evoluir sem perder controle.

1. Finalidade do contrato

Este documento fecha a etapa final do programa ENOVA LLM-first: entrada oficial em produção, rotina de operação contínua, governança de incidentes, revisão de métricas e trilha de evolução controlada. Seu objetivo é impedir que o sistema entre no ar e volte a ser tratado no improviso.

2. Princípios canônicos

- Go-live não encerra o projeto; inaugura a operação controlada.
- O primeiro dia em produção precisa de leitura mais intensa que o último dia de homologação.
- Toda melhoria pós-produção deve preservar contratos, schema, policy e telemetria.
- Incidente recorrente vale mais que opinião isolada.
- Evolução deve ser priorizada por impacto operacional e risco, não por impulso.

3. Gate final de go-live

- rollout/canary já sustentado com KPI dentro do limite
- telemetria íntegra ponta a ponta
- policy engine estável nos casos críticos
- schema validado em produção controlada
- rollback pronto e testado
- janela de monitoramento intensivo definida para D0–D3

4. Checklist de autorização

Item	Critério	Status esperado	Observação
Observabilidade	trilha completa por turno	OK	sem isso, não sobe
Memória	snapshot + facts + sobrescrita segura	OK	sem repetição cega
Roteador	GPT-4 default / GPT-5 escalado	OK	custo governado
Segurança	fallback e resposta segura prontos	OK	sem salto no escuro
Rollout	canary sustentado	OK	não pular etapa

5. Janela operacional do go-live

```
D0:
- ativar go-live com monitoramento intensivo
- revisar turnos críticos em tempo curto
- congelar mudanças não essenciais

D1–D3:
- leitura diária aprofundada
- triagem de incidentes
- ajuste fino de thresholds, prompts e policy

D4–D7:
- estabilização
- fechamento dos incidentes de maior severidade
- início da priorização de melhorias
```

6. Rotina contínua de operação

6.1 Rotina diária

- revisar KPIs principais
- ler incidentes novos e recorrentes
- inspecionar amostra de turnos críticos
- validar custo, latência e fallback rate
- abrir ação cirúrgica para qualquer desvio relevante

6.2 Rotina semanal

- fechar causas-raiz dos incidentes relevantes
- revisar regras novas a incorporar
- comparar comportamento por perfil de lead
- reavaliar thresholds do roteador e policy
- decidir backlog de evolução da semana seguinte

7. Incidentes e resposta operacional

Depois do go-live, todo problema relevante deve entrar no fluxo oficial de incidente. A resposta nunca deve pular diagnóstico, classificação e contenção.

```
Nível 1 — baixo impacto:
- corrigir no ciclo normal

Nível 2 — impacto moderado:
- priorizar correção na janela curta

Nível 3 — impacto alto:
- conter imediatamente
- acionar fallback, redução de exposição ou rollback parcial
```

8. KPIs oficiais pós-produção

- schema_valid rate
- policy violation rate

- taxa de confirmação tardia
- repetição indevida de pergunta
- latência média por turno
- custo médio por lead
- fallback rate
- incidentes por severidade
- tempo para fechar causa-raiz

9. Política de mudanças pós-go-live

- mudança urgente entra só com evidência do problema e plano claro de reversão
- mudança estrutural não entra misturada com ajuste fino
- todo ajuste de prompt relevante deve ser versionado
- toda nova regra de policy deve vir com teste e telemetria
- mudanças em memória, roteador ou schema exigem validação extra

10. Trilha oficial de evolução pós-produção

Frente	Meta	Quando abrir	Gate
E1 — refinamento de prompt	reduzir ambiguidade e repetição	após estabilização inicial	telemetria confiável
E2 — ampliação de policy	cobrir mais regras do funil	após base estável	sem aumentar falso bloqueio
E3 — multimodal	áudio, imagem, sticker, docs mais ricos	após texto estabilizado	observabilidade pronta
E4 — otimização de custo	melhor roteamento e contexto	após comportamento estável	qualidade não pode cair
E5 — GPT-5 ampliado	subir uso do modelo forte	após prova de ganho líquido	budget e KPI aprovados

11. Código-base do ciclo operacional

```
daily_ops():
  revisar_kpis()
  ler_incidentes_novos()
  auditar_turnos_criticos()
  abrir_acoes_cirurgicas()

weekly_ops():
  fechar_causa_raiz()
  revisar_backlog()
  recalibrar_thresholds()
  aprovar_frentes_da_semana()
```

12. NO-GO pós-go-live

- não abrir múltiplas frentes estruturais ao mesmo tempo
- não mexer em prompt, schema, roteador e memory juntos sem necessidade real
- não desligar telemetria para 'ganhar velocidade'
- não assumir estabilidade sem olhar incidentes e KPIs

13. Janela executiva D0–D30

- D0–D3: vigilância máxima
- D4–D7: estabilização inicial
- D8–D14: fechamento dos ajustes mais críticos
- D15–D21: leitura por padrão de falha e ganho
- D22–D30: início da evolução pós-produção com prioridade real

14. Fechamento executivo

Este contrato fecha o ciclo final do novo modelo. Com ele, a ENOVA não só entra em produção: ela entra com rotina, leitura, controle e caminho de evolução. É o documento que transforma a virada em operação adulta.

ENOVA LLM-FIRST v1

PDF 17 — Índice Mestre Consolidado + Ordem Executiva Final

Documento de fechamento da coleção: mapa de todos os PDFs, ordem exata de leitura, ordem exata de execução e sequência prática de implantação.

1. Finalidade

Este documento consolida a coleção completa do novo modelo ENOVA LLM-first. Ele existe para evitar perda de rumo, leitura fora de ordem e execução confusa. A função dele é simples: dizer o que foi produzido, para que serve cada peça e qual é a ordem correta de uso.

2. Coleção consolidada

PDF	Nome	Função	Status
1	Plano Canônico Macro	visão macro do modelo	fechado
2	Contrato de Implantação	cláusulas, escopo e gates	fechado
3	Plano Tático de Execução	frentes e microetapas	fechado
4	Backlog Mestre	épicos, dependências e ondas	fechado
5	Playbook de Implantação e Operação	Supabase, rollout e operação	fechado
6	Taxonomia, Modelo de Dados e Memória	facts, tabelas e estado	fechado
7	Ordem Executiva e Contrato Rígido	stack real + cronograma agressivo	fechado
8	Policy Engine plugável no Worker	código e fluxo de validação	fechado
9	Structured Output + Prompt Canônico	contrato do LLM	fechado
10	Roteador de Modelos	GPT-4 default / GPT-5 escalado	fechado
11	Memória Operacional do Lead	snapshot, contexto e promoção de fatos	fechado
12	Telemetria + Observabilidade	eventos, emitter e diagnóstico	fechado
13	Rollout, Shadow, Canary e Cutover	entrada controlada em operação	fechado
14	Segurança Operacional + Fail-Safes	resposta segura e proteção	fechado
15	Testes Reais + Bateria de Casos	aprovação antes do go-live	fechado
16	Go-Live + Operação Contínua	vida operacional pós-entrada	fechado

3. Ordem correta de leitura

- Primeiro bloco: PDFs 1 a 4 — visão, contrato, tática e backlog.
- Segundo bloco: PDFs 5 a 7 — base operacional, dados e ordem rígida.

- Terceiro bloco: PDFs 8 a 12 — motor real, memória, roteador e observabilidade.
- Quarto bloco: PDFs 13 a 16 — rollout, segurança, testes e operação viva.

4. Ordem correta de execução

- Passo 1: consolidar stack real, Supabase, estado e schema.
- Passo 2: subir prompt, structured output e policy engine.
- Passo 3: subir memória operacional, roteador e telemetria.
- Passo 4: ativar shadow, depois canary, depois cutover.
- Passo 5: validar bateria de casos, autorizar go-live e entrar na rotina contínua.

5. Ordem executiva curta

- D1–D2: estado, schema, prompt, policy.
- D3–D4: memória, roteador, telemetria.
- D5–D6: shadow + bateria crítica.
- D7–D10: canary e correções cirúrgicas.
- D11–D14: cutover controlado ou manutenção em canary ampliado.

6. Regra final de governança

- Não abrir múltiplas frentes estruturais ao mesmo tempo.
- Não misturar correção urgente com redesign.
- Toda mudança relevante precisa de telemetria, teste e rollback.
- Nada entra no escuro.

7. Fechamento

Este índice mestre fecha a coleção e vira o mapa oficial de implantação do novo motor. A partir daqui, o próximo passo natural já não é mais escrever documento macro: é abrir a execução real em cima desta ordem.

ENOVA LLM-FIRST v1

Contrato do Agent + Policy Engine

Documento técnico canônico para implantação do agente conversacional da Enova com GPT-4 e trilha de evolução para GPT-5.

1. Identidade do Agent

- A ENOVA é uma especialista em financiamento imobiliário no programa Minha Casa Minha Vida.
- Seu papel não é apenas conversar; seu papel é conduzir o cliente com precisão até a análise de financiamento, coletando informações essenciais de forma natural, humana e estratégica.
- Você opera dentro de um sistema com regras rígidas de negócio.
- Você pode conversar livremente, mas não pode ignorar regras obrigatórias, pular coleta de dados essenciais, prometer aprovação ou dar informações fora das diretrizes do programa.

2. Objetivo Global

- Entender o perfil do cliente.
- Classificar corretamente dentro das regras do programa.
- Coletar todas as informações necessárias.
- Levar o cliente até o envio de documentos ou para a próxima ação concreta.

3. Princípios Operacionais

- Nunca invente informação.
- Nunca pule etapas críticas.
- Sempre valide informações ambíguas.
- Sempre conduza o cliente para o próximo passo.
- Nunca finalize sem ação clara.
- Nunca deixe o cliente solto.

4. Regras de Negócio Obrigatórias

- Estado civil: casado no civil implica processo obrigatoriamente em conjunto; união estável pode ser solo ou conjunto conforme o contexto; solteiro pode seguir solo ou em composição.
- Renda: renda baixa deve acionar sugestão de composição de renda; nunca encerrar sem antes tentar viabilizar.
- Autônomo: sempre perguntar sobre Imposto de Renda; sem IR, considerar capacidade reduzida e sugerir alternativas viáveis.
- Estrangeiro: exige RNM válido; sem RNM válido, não avançar.
- CTPS: 36 meses melhora condição, mas não pode travar o fluxo.

- Dependente: só perguntar quando realmente fizer sentido.
- Nunca prometer aprovação, valor exato ou condição final; sempre reforçar que tudo depende da análise do perfil e da avaliação do banco.

5. Comportamento Conversacional

- Linguagem natural, humana e acessível.
- Nunca robótico e nunca técnico demais.
- Sempre claro, direto e com leve direcionamento.
- Se o cliente sair do assunto, responder e trazer de volta para o fluxo.
- Se o cliente travar, oferecer opções simples.
- Sempre conduzir.

6. Formato de Saída Obrigatório

O modelo deve responder internamente em estrutura estável, permitindo validação pelo Worker antes de persistir ou avançar o caso.

```
{
  "reply_text": "mensagem para o cliente",
  "intent": "coleta_dado | resposta_duvida | confirmacao | direcionamento",
  "facts_extracted": {
    "estado_civil": null,
    "renda": null,
    "regime_trabalho": null,
    "autonomo_tem_ir": null,
    "nacionalidade": null,
    "rnm_status": null
  },
  "facts_updated": [],
  "current_objective": "o que está tentando coletar agora",
  "next_step": "próxima ação esperada",
  "needs_confirmation": false,
  "confidence": 0.0,
  "policy_flags": []
}
```

7. Regras de Segurança Anti-Drift

- Se houver dúvida, marcar needs_confirmation = true.
- Se houver contradição, não avançar e pedir confirmação.
- Se faltar informação obrigatória, não avançar.
- Se uma regra for violada, corrigir o fluxo antes de responder ao cliente.

8. Estratégia de Raciocínio

- O agent não decide livremente o fluxo.
- Ele interpreta o cliente, extrai sinais e sugere o próximo passo.

- O sistema valida a decisão.
- Em insegurança, deve sinalizar e evitar conclusão forçada.

9. Compatibilidade de Modelo

GPT-4 é o baseline de implantação. O contrato já nasce compatível com upgrade futuro para GPT-5, sem mudança estrutural do payload, do estado ou da camada de policy.

- Com GPT-4: usar temperatura baixa e validação externa mais firme.
- Com GPT-5: ganhar maior consistência, melhor interpretação ambígua e menor dependência de correção externa.

10. Configuração Recomendada

```
model: GPT-4 baseline (com trilha de upgrade para GPT-5)
temperature: 0.3
max_tokens: 500–800
top_p: 1
frequency_penalty: 0
presence_penalty: 0
```

11. Policy Engine Plugável no Worker

A camada de policy não substitui o agent. Ela valida se o agent está respeitando as regras obrigatórias antes de avançar o estado do lead.

```
function policyCheck(state, llmOutput) {
  const violations = [];

  if (state.regime_trabalho === "autonomo" && state.autonomo_tem_ir == null) {
    violations.push("PERGUNTAR_IR");
  }

  if (state.estado_civil === "casado_civil" && state.processo !== "conjunto") {
    violations.push("FORCAR_CONJUNTO");
  }

  if (typeof state.renda === "number" && state.renda < 3000 && state.processo === "solo") {
    violations.push("SUGERIR_COMPOSICAO");
  }

  if (state.nacionalidade === "estrangeiro" && state.rnm_status !== "valido") {
    violations.push("BLOQUEAR_AVANCO");
  }

  return violations;
}
```

12. Orquestração da Decisão

```
const violations = policyCheck(state, llmOutput);

if (violations.length > 0) {
  return corrigirFluxoComBaseNasRegras(state, llmOutput, violations);
}

return seguirFluxo(state, llmOutput);
```

13. Resultado Esperado

- LLM como motor principal de conversa.
- Worker como orquestrador e policy gateway.
- Supabase como fonte única da verdade do lead.
- Contrato estável para operar com GPT-4 agora e evoluir para GPT-5 depois sem reescrever a arquitetura.

ENOVA LLM-FIRST v1

PDF 8 — Policy Engine plugável no Worker (código, fluxo e integração)

Documento operacional para implantação do validador mínimo de regras no novo modelo LLM-first.

1. Objetivo

Este documento fecha a peça operacional que faltava entre o contrato do agent e a execução real: um Policy Engine mínimo, previsível e plugável no Worker, capaz de validar fatos críticos, impedir avanço indevido e forçar coleta obrigatória sem recriar o mecânico pesado.

2. Função do Policy Engine

- Receber o estado estruturado atual do lead e o output estruturado do LLM.
- Aplicar regras obrigatórias de negócio.
- Detectar violações, pendências e bloqueios.
- Corrigir o rumo antes de persistir ou responder ao cliente.
- Registrar evento de policy e manter telemetria auditável.

3. Contrato de entrada

3.1 Estado mínimo esperado

```
{
  "lead_id": "string",
  "estado_civil": "solteiro | uniao_estavel | casado_civil | null",
  "processo": "solo | conjunto | composicao_familiar | null",
  "renda_principal": 0,
  "regime_trabalho": "clt | autonomo | servidor | aposentado | null",
  "autonomo_tem_ir": true,
  "nacionalidade": "brasileiro | estrangeiro | null",
  "rnm_status": "valido | invalido | ausente | null",
  "dependente_qtd": 0,
  "ctps_36": true,
  "facts_confirmados": {},
  "facts_ambiguos": [],
  "pendencias_obrigatorias": []
}
```

3.2 Output mínimo esperado do LLM

```
{
  "reply_text": "mensagem ao cliente",
  "intent": "coleta_dado | resposta_duvida | confirmacao | direcionamento",
  "facts_extracted": {
    "estado_civil": null,
    "processo": null,
    "renda_principal": null,
    "regime_trabalho": null,
    "autonomo_tem_ir": null,
    "nacionalidade": null,
```

```
    "rnm_status": null
  },
  "facts_updated": [],
  "current_objective": "coletar estado civil",
  "next_step": "perguntar estado civil",
  "needs_confirmation": false,
  "confidence": 0.0,
  "policy_flags": []
}
```

4. Regras mínimas obrigatórias

ID	Condição	Ação obrigatória	Tipo
R1	estado_civil = casado_civil	forçar processo = conjunto	roteamento
R2	regime_trabalho = autonomo	perguntar IR se ainda ausente	coleta obrigatória
R3	renda_principal < 3000 e processo = solo	sugerir composição	sugestão obrigatória
R4	nacionalidade = estrangeiro e rnm_status != valido	bloquear avanço	bloqueio
R5	output com contradição factual	pedir confirmação	consistência
R6	fato crítico ausente	não avançar objetivo	gate leve

5. Fluxo de execução no Worker

```
entrada do cliente
    ↓
carregar estado no Supabase
    ↓
montar prompt + estado + memória
    ↓
chamar OpenAI
    ↓
receber output estruturado do LLM
    ↓
rodar policyEngine(state, llmOutput)
    ↓
se houver bloqueio/pendência:
    corrigir resposta + manter objetivo
senão:
    persistir fatos + atualizar estado + responder
    ↓
registrar policy_event + turn log
```

6. Código real plugável no Worker

6.1 Função principal do policy engine

```
export function runPolicyEngine(state, llmOutput) {
  const violations = [];
  const requiredQuestions = [];
  const forcedUpdates = {};
  let blockAdvance = false;
  let needsConfirmation = Boolean(llmOutput?.needs_confirmation);
```

```
const facts = {
  ...state,
  ...(llmOutput?.facts_extracted || {})
};

if (facts.estado_civil === "casado_civil") {
  if (facts.processo !== "conjunto") {
    forcedUpdates.processo = "conjunto";
    violations.push({
      code: "FORCAR_CONJUNTO",
      severity: "high",
      reason: "Casado civil exige processo em conjunto."
    });
  }
}

if (facts.regime_trabalho === "autonomo" && facts.autonomo_tem_ir == null) {
  requiredQuestions.push("autonomo_tem_ir");
  violations.push({
    code: "PERGUNTAR_IR",
    severity: "medium",
    reason: "Autônomo exige pergunta obrigatória sobre IR."
  });
}

if (
  typeof facts.renda_principal === "number" &&
  facts.renda_principal < 3000 &&
  facts.processo === "solo"
) {
  violations.push({
    code: "SUGERIR_COMPOSICAO",
    severity: "medium",
    reason: "Renda solo baixa deve disparar sugestão de composição."
  });
}

if (facts.nacionalidade === "estrangeiro" && facts.rnm_status !== "valido") {
  blockAdvance = true;
  violations.push({
    code: "BLOQUEAR_AVANCO_RNM",
    severity: "critical",
    reason: "Estrangeiro sem RNM válido não pode avançar."
  });
}

if (hasCriticalContradiction(state, llmOutput?.facts_extracted || {})) {
  blockAdvance = true;
  needsConfirmation = true;
  violations.push({
    code: "CONTRADICAO_FATUAL",
    severity: "high",
    reason: "O LLM extraiu fato conflitante com o estado confirmado."
  });
}

return {
  ok: violations.length === 0,
  blockAdvance,
  needsConfirmation,
  violations,
  requiredQuestions,
```

```
    forcedUpdates
  };
}
```

6.2 Detector mínimo de contradição

```
export function hasCriticalContradiction(state, extracted) {
  const criticalKeys = [
    "estado_civil",
    "processo",
    "nacionalidade",
    "rnm_status",
    "regime_trabalho",
    "autonomo_tem_ir"
  ];

  for (const key of criticalKeys) {
    const oldVal = state?.[key];
    const newVal = extracted?.[key];
    if (
      oldVal != null &&
      newVal != null &&
      oldVal !== newVal
    ) {
      return true;
    }
  }

  return false;
}
```

6.3 Orquestração no pipeline do Worker

```
const llmOutput = await callOpenAIWithStructuredOutput({
  systemPrompt,
  state,
  memory,
  userMessage
});

const policy = runPolicyEngine(state, llmOutput);

const mergedState = {
  ...state,
  ...(llmOutput.facts_extracted || {}),
  ...(policy.forcedUpdates || {})
};

const finalReply = buildFinalReply({
  state,
  mergedState,
  llmOutput,
  policy
});

await saveTurn({
  leadId: state.lead_id,
  userMessage,
  llmOutput,
  policy,
  finalReply
});
```

```
if (!policy.blockAdvance) {
  await persistLeadState(mergedState);
}

return finalReply;
```

6.4 Builder da resposta final

```
export function buildFinalReply({ state, mergedState, llmOutput, policy }) {
  if (policy.blockAdvance) {
    if (policy.violations.some(v => v.code === "BLOQUEAR_AVANCO_RNM")) {
      return {
        reply_text:
          "Para seguir com a análise, preciso confirmar seu RNM válido. " +
          "Sem isso eu não consigo avançar com segurança.",
        next_step: "confirmar_rnm",
        policy_flags: policy.violations.map(v => v.code)
      };
    }

    if (policy.needsConfirmation) {
      return {
        reply_text:
          "Quero confirmar um ponto para não seguir errado. " +
          "Você pode me confirmar essa informação?",
        next_step: "confirmacao",
        policy_flags: policy.violations.map(v => v.code)
      };
    }
  }

  if (policy.requiredQuestions.includes("autonomo_tem_ir")) {
    return {
      reply_text:
        "Perfeito. Como você atua como autônomo, me confirma uma coisa: " +
        "você declara Imposto de Renda?",
      next_step: "perguntar_ir",
      policy_flags: policy.violations.map(v => v.code)
    };
  }

  if (policy.violations.some(v => v.code === "SUGERIR_COMPOSICAO")) {
    return {
      reply_text:
        llmOutput.reply_text +
        "\n\nPelo valor da renda, pode ser estratégico avaliar composição com alguém da família
" +
        "ou com parceiro, para aumentar a força da análise.",
      next_step: llmOutput.next_step,
      policy_flags: policy.violations.map(v => v.code)
    };
  }

  return {
    reply_text: llmOutput.reply_text,
    next_step: llmOutput.next_step,
    policy_flags: policy.violations.map(v => v.code)
  };
}
```

7. Persistência mínima no Supabase

```
-- tabela principal de estado
enova_lead_state_v2

-- log por turno
enova_turns

-- eventos do policy engine
enova_policy_events
  id
  lead_id
  turn_id
  code
  severity
  reason
  block_advance
  created_at
```

8. Regras de implantação

- Primeira versão entra em shadow mode por alguns dias, sem mandar a resposta policy-corrigida ao cliente quando ainda houver dúvida estrutural.
- O primeiro corte operacional deve cobrir apenas regras de alto valor e alta segurança.
- Não tentar migrar todas as regras do funil no primeiro dia.
- Toda nova regra entra com: condição, ação, severidade, evidência e teste.
- Worker continua como orquestrador; não criar microserviço extra agora.

9. Janela realista de execução

- D1–D2: subir função, persistência e telemetria mínima.
- D3–D4: integrar ao output estruturado do agent.
- D5–D6: validar regras R1–R4 com casos reais controlados.
- D7–D9: ampliar cobertura e acertar builder final.
- D10–D14: shadow + correções + corte de go-live.

10. Fechamento

Este policy engine não substitui o cérebro conversacional. Ele substitui a parte dura de garantia. Ou seja: o LLM conduz, mas não fica livre para descumprir regra crítica. Esse é o bloco que permite escalar para áudio, multimodalidade e conversa natural sem virar bagunça.

PACOTE COMPLEMENTAR

ENOVA 2 — Documento Canônico Geral e Regra de Envio

Documento-base para abrir novas abas sem reenviar o pacote completo. Este arquivo define a hierarquia dos contratos, a regra oficial de uso e quais PDFs devem acompanhar cada frente.

1. Objetivo deste documento

Este PDF existe para reduzir fricção operacional. A partir dele, você não precisa mais reenviar todos os contratos antigos e novos em cada aba. Ele centraliza o que manda, o que complementa e em quais cenários cada arquivo deve ser enviado.

A ENOVA 2 nasce como sistema novo, em repositório novo, com arquitetura LLM-first disciplinada por contrato. As 35 mil linhas da ENOVA 1 não entram como fundação. O que entra é a inteligência de negócio: regras, microregras, funil, taxonomia, persistência útil, integrações e aprendizado acumulado.

2. Hierarquia oficial dos documentos

A ordem de autoridade documental da ENOVA 2 fica assim:

- **Nível 1 — Documento Canônico Geral**: manda na visão macro, no recorte do projeto, na regra de envio de arquivos e na hierarquia entre contratos.
- **Nível 2 — Backlog Mestre + Ordem Executiva**: define fase ativa, frente ativa, contrato ativo, próximo passo e critérios de bloqueio.
- **Nível 3 — Contratos por frente**: detalham o que cada subsistema pode fazer, o que não pode fazer e como provar aceite.
- **Nível 4 — Contratos legados úteis da ENOVA 1**: entram como fonte de regra de negócio, nunca como carta branca para reproduzir arquitetura antiga de speech.

3. Regra de envio para novas abas

Situação	Arquivos mínimos para enviar	Observação operacional
Abertura de aba estratégica ou decisão de arquitetura	01 Canônico Geral + 02 Backlog Mestre + Ordem Executiva	Com esses dois arquivos já é possível abrir, priorizar e manter o rumo.
Trabalho em frente específica	01 + 02 + contrato da frente	Enviar só o contrato específico do subsistema em pauta.
Execução técnica com	01 + 02 + contrato da frente +	O legado entra apenas como

Copilot/Codex	eventual contrato legado útil	fonte de regra, não como base de implementação.
Discussão de áudio/multimodal	01 + 02 + 06 Áudio + 07 Supabase	Áudio sempre conversa com persistência.
Discussão de Meta/WhatsApp	01 + 02 + 08 Meta/WhatsApp + 07 Supabase	Canal sem persistência inteligente gera drift operacional.

4. O que continua valendo dos 19 PDFs existentes

Os 19 PDFs já produzidos continuam valendo como fonte de regra e histórico decisório. Eles não são descartados. Porém, na ENOVA 2, eles passam a ter papéis diferentes:

- **Continuam soberanos como fonte de negócio**: regras do MCMV, microregras do funil, critérios de aceite por bloco, taxonomia, rollout, telemetria e structured output já pactuados.
- **Deixam de ser soberanos como fundação de speech**: a nova arquitetura não reproduz a cadeia de fallback, rawArr, helper e bridge da ENOVA 1.
- **Viram referência cruzada**: quando houver dúvida sobre regra de negócio, prevalece o legado contratual; quando houver dúvida sobre arquitetura nova, prevalece este pacote complementar.

5. Regra macro da ENOVA 2

- Uma única camada pode falar por turno.
- O Core Mecânico decide stage, gate, parse, nextStage, persistência e business rules.
- O Speech Engine monta a surface final obedecendo o stage atual e a política de transição.
- Nenhuma ponte coleta o slot do próximo stage por acidente.
- Nenhum fallback genérico pode vencer coleta crítica.
- Se o cliente respondeu o stage atual e existe nextStage de coleta, a continuidade precisa ser deliberada: sem silêncio e sem duplicação.
- Áudio, texto, contexto e múltiplos sinais do mesmo turno devem virar dados estruturados antes da persistência final.

6. Pacote complementar criado agora

Código	Arquivo	Finalidade
01	Documento Canônico Geral e Regra de Envio	Define a hierarquia e reduz a necessidade de reenviar tudo.
02	Backlog Mestre + Ordem Executiva	Controla fase ativa, critérios de bloqueio, sequência e rollout.
03	Contrato do Core Mecânico + Policy Engine	Formaliza a nova autoridade de decisão.
04	Contrato do Speech Engine +	Mata a disputa de camadas de

	Surface Única	fala.
05	Contrato de Contexto, Slots e Memória Viva	Explica como lidar com cliente que fala várias coisas ao mesmo tempo.
06	Contrato de Áudio e Multimodalidade	Padroniza transcrição, extração e resposta por voz.
07	Contrato do Supabase Adapter e Persistência Inteligente	Amarra memória, estado, evidência e integração com estrutura existente.
08	Contrato do Canal Meta/WhatsApp	Formaliza entrada, saída, idempotência, mídia e operação do canal.
09	Contrato da Analista MCMV Virtual e Memorial do Programa	Transforma o LLM em analista virtual, não em bot scriptado.

7. Regra final de governança

Nenhuma frente da ENOVA 2 começa sem que o contrato da frente esteja ativo no Backlog Mestre + Ordem Executiva.

Nenhum contrato específico pode contradizer este documento. Se contradizer, este documento manda até que o pacote seja revisado oficialmente.

PACOTE COMPLEMENTAR

ENOVA 2 — Backlog Mestre e Ordem Executiva

Documento operacional para conduzir o projeto sem se perder. Define sequência, gates, bloqueios, entregáveis e ordem oficial de implantação.

1. Como este documento deve ser usado

Este PDF é o painel de comando do projeto. Em qualquer aba técnica, a pergunta central deve ser: qual é a frente ativa agora, qual contrato está ativo agora, o que está bloqueado e qual o próximo passo único?

Nada entra em execução fora da ordem deste documento, salvo revisão explícita do canônico geral.

2. Frentes da ENOVA 2 e ordem oficial

Ordem	Frente	Objetivo	Bloqueios para abrir a frente seguinte
F1	Macro + Scaffold	Definir repo, arquitetura base, serviços, convenções e política de branches.	Só fecha quando houver blueprint validado e estrutura mínima pronta.
F2	Core Mecânico 2	Novo motor de decisão soberano, sem speech concorrente.	Precisa decidir stage/nextStage/persistência com previsibilidade.
F3	Speech Engine	Uma única surface final por turno, com política de transição.	Não pode haver duplicação, silêncio ou coleta indevida.
F4	Contexto + Extração + Memória Viva	Capturar várias informações do mesmo turno e transformá-las em dados estruturados.	Precisa salvar com confiança e confirmação controlada.
F5	Supabase Adapter	Persistência nova sobre a base atual do Supabase, sem corromper a ENOVA 1.	Precisa provar integridade e compatibilidade.
F6	Áudio + Multimodal	Transcrever, interpretar, salvar e responder com lógica multimodal.	Só abre depois de texto puro estável.

F7	Meta/WhatsApp	Canal produtivo, mensagens, mídias, idempotência e observabilidade.	Só abre com persistência estável.
F8	Analista MCMV Virtual	Camada cognitiva de negócio ligada ao memorial e às regras do programa.	Só abre com core/speech/contexto estáveis.
F9	Shadow / Canary / Cutover	Implantação segura e migração gradual.	Só abre com as frentes anteriores testadas.

3. Backlog mestre por prioridade

- **Prioridade 0**: abrir repo novo, estrutura de pastas, padrões de naming, variáveis de ambiente, políticas de teste e CI mínima.
- **Prioridade 1**: modelar o Core Mecânico 2 com contratos por stage, sem acoplar fala.
- **Prioridade 2**: modelar o Speech Engine com surface única e política explícita para turnos de transição.
- **Prioridade 3**: definir o schema de extração estruturada do turno: slots, sinais, contexto, objeções, pendências e evidências.
- **Prioridade 4**: criar a camada de persistência sobre Supabase com nomes novos, logs novos e trilho de compatibilidade.
- **Prioridade 5**: só então plugar áudio, Meta e rollout.

4. Critérios duros de bloqueio

- Sem contrato da frente, não começa implementação.
- Sem smoke da frente, não promove para a próxima frente.
- Sem previsibilidade operacional, áudio e canal ficam bloqueados.
- Sem persistência íntegra, nenhuma memória viva entra em produção.
- Sem observabilidade mínima, não existe shadow/canary/cutover.

5. Entregáveis mínimos por frente

Frente	Entregável mínimo	Prova exigida
F2 Core	decisor funcional de texto puro	transições corretas + persistência correta
F3 Speech	surface única por turno	sem duplicação, sem silêncio, sem fallback genérico crítico
F4 Contexto	extração de múltiplos sinais no mesmo turno	JSON estruturado + reconciliação mecânica
F5 Supabase	adapter e tabelas/colunas	gravação idempotente +

	novas ou namespace novo	replay seguro
F6 Áudio	pipeline STT -> extração -> decisão -> resposta	turno por áudio salva e anda no funil
F7 Meta	canal produtivo controlado	mensagem entra, sai, persiste e é rastreável

6. Regra operacional para novas abas

Se a aba for estratégica ou de coordenação: enviar 01 + 02.

Se a aba for de construção de frente: enviar 01 + 02 + contrato da frente.

Se a aba for de execução Copilot/Codex: enviar também o handoff específico e, quando necessário, o contrato legado da ENOVA 1 que contém a regra de negócio relevante.

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato do Core Mecânico e Policy Engine

Contrato do motor soberano da ENOVA 2. Este documento determina como o sistema decide, persiste e anda no funil sem depender da fala.

1. Missão do Core Mecânico

O Core Mecânico é a autoridade única sobre decisão operacional. Ele recebe sinais estruturados do turno, valida o que é confiável, aplica microregras do MCMV, decide o que foi coletado, o que ainda falta e qual é o próximo objetivo do fluxo.

2. O que o Core Mecânico pode fazer

- Ler estado atual do lead.
- Ler sinais extraídos do turno.
- Aplicar regra de negócio e microregra do MCMV.
- Confirmar ou rejeitar slots.
- Calcular nextStage.
- Persistir estado oficial.
- Emitir instruções para o Speech Engine.

3. O que o Core Mecânico não pode fazer

- Não escreve a fala final ao cliente.
- Não produz resposta natural longa.
- Não improvisa consultoria fora de hora.
- Não depende de rawArr para parecer inteligente.
- Não aceita slot só porque o LLM parece convicto; precisa de regra e confiança.

4. Modelo de entrada e saída

Entrada	Descrição
state_snapshot	estado consolidado do lead antes do turno
turn_extract	extração estruturada do texto/áudio/imagem
channel_context	origem do turno, mídia, metadados, ids
policy_context	regras ativas, limites, travas e memorandos do programa

Saída mínima do Core Mecânico:

- confirmed_slots
- rejected_slots
- pending_slots
- stage_after
- next_objective
- required_confirmation
- persist_ops
- speech_intent

5. Política de decisão

- Se o cliente entregar múltiplas informações no mesmo turno, o Core tenta capturar tudo o que for validável, sem perder o objetivo do stage atual.
- Se a informação pertence a stage futuro, ela pode ser registrada como sinal ou pré-slot, mas só vira slot oficial quando a regra permitir.
- Se houver conflito entre regras, prevalece a regra mais restritiva até confirmação.

6. Relação com as microregras do MCMV

As microregras do MCMV continuam vindo da ENOVA 1 e dos contratos já aprovados. A ENOVA 2 não reinventa regra de negócio. Ela apenas muda a arquitetura que aplica e conversa sobre essa regra.

7. Teste mínimo de aceite do Core

- Textos longos com múltiplos sinais não podem quebrar o nextStage.
- O Core deve conseguir salvar o que já sabe e dizer o que ainda falta.
- Nenhuma decisão de negócio deve depender de frase pronta do Speech Engine.

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato do Speech Engine e Surface Única

Contrato da camada que fala com o cliente. A função desta camada é eliminar disputa entre rawArr, fallback, bridge, helper e prompt de retorno.

1. Missão do Speech Engine

O Speech Engine recebe a intenção oficial do Core Mecânico e monta a única surface final autorizada a sair para o cliente. Toda resposta do turno passa por ele.

2. Regra inviolável

- Uma única camada fala por turno.
- Se o stage foi respondido e existe nextStage de coleta, a continuidade é deliberada: sem silêncio e sem duplicação.
- Ponte nunca coleta slot do próximo stage por acidente.

3. Entradas do Speech Engine

- stage atual
- stage seguinte
- speech_intent do Core
- slots confirmados/rejeitados
- flags de confirmação pendente
- contexto do canal e do cliente

4. Saídas permitidas

- pergunta do stage atual
- reanchor do stage atual
- confirmação curta + próxima pergunta em transição autorizada
- resposta de dúvida dentro do contrato do stage
- fallback controlado e parser-compatible

5. Saídas proibidas

- duas perguntas do mesmo slot
- pergunta híbrida que mistura stage atual e futuro

- fallback genérico tipo “Pode continuar” em coleta crítica
- rawArr ou helper mecânico vazando direto para produção

6. Política de transição

Em transições de coleta, o sistema deve escolher explicitamente entre dois modos válidos e apenas um deles:

- Modo A — confirmação curta da decisão e, no mesmo turno, pergunta canônica do próximo stage.
- Modo B — pergunta direta do próximo stage sem confirmação redundante quando a confirmação for desnecessária.
- Não existe modo C com confirmação seca seguida de silêncio.
- Não existe modo D com pergunta do próximo stage repetida duas vezes.

7. Relação com o LLM

- O LLM continua soberano apenas na fala.
- O LLM não inventa stage, não muda regra, não troca nextStage.
- Toda fala gerada pelo LLM é validada contra o contrato do stage antes de sair.
- Quando o LLM falhar, o Speech Engine cai para fallback controlado específico, nunca para genérico crítico.

8. Teste mínimo de aceite do Speech Engine

- zero silêncio indevido
- zero duplicação de pergunta
- zero coleta fora do stage
- zero fallback genérico crítico
- surface parser-compatible em turnos de coleta

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato de Contexto, Extração de Slots e Memória Viva

Contrato da camada que transforma conversa livre em dados estruturados, sem perder nuance nem quebrar o funil.

1. Missão desta camada

A ENOVA 2 precisa entender cliente falando como gente, não como formulário. Isso exige uma camada capaz de extrair do mesmo turno dados de funil, objeções, restrições, preferências, contexto familiar, intenção e sinais de risco.

2. Objetos mínimos extraídos por turno

Objeto	Exemplo
slots_candidatos	estado_civil=solteiro; renda_aprox=3000
signals	quer_visita=true; medo_documentos=true
context_notes	cliente está desempregado há pouco tempo, mas compõe com mãe
objections	não quer mandar docs por celular
cross_stage_info	falou de FGTS antes da hora, mas isso é útil depois

3. Regra de reconciliação

- Nem tudo que o cliente fala vira slot oficial.
- O extractor pode capturar muita coisa; quem oficializa é o Core Mecânico.
- Informação útil fora de hora pode ser guardada como contexto e pré-slot.
- Quando houver ambiguidade real, o sistema confirma antes de consolidar.

4. Memória viva vs memória estrutural

- **Memória viva**: resumo operativo curto do que já foi dito, útil para manter naturalidade e evitar repetição burra.
- **Memória estrutural**: slots confirmados, flags de regra, pendências, sinais salvos e histórico de decisão.
- **Memória fria**: fatos históricos relevantes para o dossiê e para futuras etapas.

5. Regra para múltiplas informações no mesmo turno

Se o cliente disser várias coisas ao mesmo tempo, a ENOVA 2 não deve fingir que ouviu só uma. Ela deve:

- capturar tudo o que for relevante
- oficializar apenas o que a regra permitir
- responder priorizando o objetivo do fluxo
- manter em contexto o que será útil na frente

6. Critério de aceite

- o sistema precisa evitar repetições burras
- precisa conseguir usar informação dita antes sem quebrar soberania mecânica
- precisa salvar contexto auditável e reaproveitável

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato de Áudio e Multimodalidade

Contrato para recebimento, transcrição, interpretação, persistência e resposta por áudio, sem romper o funil nem a auditabilidade.

1. Objetivo

A ENOVA 2 deve receber áudio do cliente, transcrever, extrair informação útil do funil, salvar no Supabase e seguir normalmente. Mais à frente, também deve responder com áudio natural quando a operação permitir.

2. Pipeline oficial do áudio

Etapa	Descrição
1. Ingestão	receber mídia, metadata, id da mensagem e vínculo com o lead
2. STT	transcrever com confiança, timestamps e idioma
3. Normalização	limpar ruído, filler e variações coloquiais
4. Extração estruturada	capturar slots, sinais, objeções e contexto
5. Reconciliação mecânica	decidir o que entra como oficial
6. Persistência	salvar áudio, transcrição, extração e decisão
7. Resposta	texto ou áudio, conforme política do canal

3. Regras duras

- Transcrição bruta e versão normalizada devem ser auditáveis.
- Dados do funil extraídos do áudio precisam ser rastreáveis até a transcrição de origem.
- Se a confiança for baixa, a ENOVA 2 pede confirmação em vez de consolidar dado crítico.
- Áudio não pode virar caixa-preta sem replay.

4. Resposta por áudio

- Deve reutilizar a mesma intenção do Speech Engine.
- Não pode falar algo diferente da versão textual autorizada.
- A fala por áudio é apresentação, não mudança de regra.

5. Dependência com Supabase

Áudio só entra depois que a persistência inteligente estiver contratada. Sem adapter sólido, áudio vira desordem e perda de rastreabilidade.

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato do Supabase Adapter e Persistência Inteligente

Contrato da amarração mais delicada da ENOVA 2: como usar o Supabase atual sem herdar a confusão de arquitetura da ENOVA 1.

1. Missão do Supabase Adapter

Separar o cérebro conversacional da escrita física no banco. A ENOVA 2 pode pensar livre, mas só grava por meio de um adapter contratual, idempotente, auditável e compatível com a base existente.

2. Regra de ouro

- Reaproveitar Supabase, sim. Reaproveitar sem contrato, não.
- A ENOVA 2 não deve sair escrevendo nas mesmas colunas críticas da ENOVA 1 sem namespace, camada de compatibilidade ou política de migração.

3. Estratégia recomendada

- Criar tabelas auxiliares ou namespace lógico `enova2_*` para memória viva, eventos de turno, transcrição, extração e telemetria nova.
- Manter um adapter de projeção para popular apenas os campos oficiais que a operação realmente precisa.
- Guardar evidência do turno antes da consolidação final.

4. Objetos de persistência sugeridos

Objeto	Finalidade
lead_state_v2	estado oficial do lead na ENOVA 2
turn_events_v2	cada turno bruto, normalizado e decidido
slot_evidence_v2	provas de por que um slot foi aceito
audio_assets_v2	vínculo mídia/transcrição/turno
memory_runtime_v2	resumo vivo do lead
projection_bridge_v2	espelho para o que ainda precisa conversar com a ENOVA 1

5. Regras de escrita

- Toda escrita deve ser idempotente.
- Toda atualização relevante deve guardar `why` ou referência de evidência.
- Reprocessamento do mesmo turno não pode corromper estado.
- Campos derivados e campos fontes não devem se confundir.

6. Critério de aceite

- replay seguro de turno
- consistência entre extração e projeção
- isolamento suficiente para evoluir a ENOVA 2 sem quebrar a ENOVA 1

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato do Canal Meta/WhatsApp e Operação de Mensageria

Contrato do canal produtivo. Define como a ENOVA 2 recebe, responde, correlaciona mensagens, mídias e estados no WhatsApp.

1. Missão do canal

Receber eventos do WhatsApp, correlacionar com leads, garantir idempotência, acionar a pipeline correta e entregar a resposta certa sem duplicação, atraso indevido ou perda de rastreabilidade.

2. Responsabilidades do canal

- ingestão de texto, áudio, imagem, documento e localização
- normalização de payloads Meta
- idempotência por message_id
- roteamento para transcrição/extração/decisão
- envio da resposta textual ou de áudio
- logs mínimos para diagnóstico

3. Regras duras

- Nenhuma mensagem deve ser processada duas vezes como se fosse nova.
- Mídia e metadados precisam ser vinculados ao turno.
- A resposta enviada ao cliente deve corresponder exatamente à surface final aprovada pelo Speech Engine.
- O canal não decide conteúdo; o canal entrega.

4. Casos especiais

- áudio com múltiplos dados do funil
- cliente que manda vários áudios seguidos
- cliente que mistura documento e dúvida no mesmo turno
- cliente que responde fora de ordem

5. Critério de aceite

- entrada confiável

- envio confiável
- sem duplicação de mensagem
- sem perda de vínculo entre mídia, turno e estado

PACOTE COMPLEMENTAR

ENOVA 2 — Contrato da Analista MCMV Virtual e do Memorial do Programa

Contrato que transforma a ENOVA 2 em analista do MCMV: natural, contextual, investigativa, mas obediente às regras do programa e ao estágio operacional.

1. Missão

A ENOVA 2 não deve ser só um robô de fases nem um “robô inteligente” genérico. Ela deve funcionar como analista virtual do MCMV: entende o cliente, percebe contexto, conecta múltiplos sinais, faz perguntas úteis e sabe quando aprofundar, sem abandonar as regras do programa.

2. Memorial do programa como fonte de inteligência

- O memorial do programa e a base documental do MCMV entram como conhecimento consultivo controlado.
- A analista pode formular perguntas adicionais quando o tipo de cliente exigir mais segurança, mais prova ou melhor enquadramento.
- Essas perguntas adicionais não podem quebrar o funil; devem ser emitidas como perguntas auxiliares autorizadas ou como coleta contextual com reconciliação posterior.

3. O que diferencia esta camada do bot antigo

Bot antigo	Analista virtual ENOVA 2
espera uma resposta por vez	consegue entender múltiplos dados no mesmo turno
reage ao script	interpreta cenário, contexto e pendências
fala mecânica rígida	fala natural sob contrato
perde nuance de áudio	converte áudio em contexto e dados
repete pergunta facilmente	lembra o que já ouviu e o que ainda falta

4. Regras duras da analista

- não promete aprovação
- não troca regra do programa por simpatia
- não inventa documento exigido
- não pula coleta crítica
- não conversa bonito se isso quebrar a operação

5. Critério de aceite

- pergunta melhor que o bot antigo
- interpreta melhor que o script antigo
- continua obediente ao Core Mecânico e ao Speech Engine